

Appunti di Ingegneria del Software

Appunti basati sulle lezioni di Davide Falessi

A.A. 2025/2026

Indice

1	Teoria della Misurazione e Scale di Misura	9
1.1	Introduzione alla Misurazione del Software	9
1.1.1	Definizioni Formali	9
1.1.2	Proprietà di una Metrica di Qualità	9
1.2	Tassonomia delle Scale di Misura	10
1.2.1	Scala Nominale e Ordinale	10
1.2.2	Scale Numeriche (Intervallo e Rapporto)	10
1.3	Modelli di Qualità: Lo Standard ISO/IEC 25010	11
1.4	Introduzione alle Metriche Object-Oriented	11
2	GQM+S, Stima dei Costi e Planning Poker	12
2.1	GQM+Strategies (Goal-Question-Metric + Strategies)	12
2.1.1	Il Reference Process (I 6 Step del GQM+S)	12
2.2	Introduzione alla Stima dei Costi	13
2.2.1	Componenti del Costo ed Overheads	13
2.2.2	Il Ruolo del Linguaggio di Programmazione	14
2.3	Tecniche di Stima	14
2.4	Planning Poker	14
2.4.1	Dinamiche e Regole del Gioco	15
2.4.2	Vantaggi e Consigli Pratici (Tips)	15
3	Introduzione all'Architettura Software	16
3.1	Cos'è un'Architettura Software	16
3.1.1	Architetture Intended vs Unintended	16
3.2	Il Modello 4+1 di Kruchten	17
3.3	Progettazione e Decision-Making	17
3.3.1	L'importanza dei Requisiti Non Funzionali	17
3.3.2	Trade-offs e il ruolo dell'Architetto	18
3.4	Documentazione: Viste e Punti di Vista	18
3.5	Software Product Lines (SPL)	18
3.6	Stili Architettureali	19
4	Debito Tecnico e Code Smells	20
4.1	Il Debito Tecnico (Technical Debt)	20
4.1.1	Cause e Conseguenze	20
4.1.2	Strategie di Gestione	21
4.2	Introduzione ai Code Smells	21
4.3	Catalogo dei "Worst Smells"	21
4.3.1	Esempi di Worst Smells	22

5	Machine Learning e Software Analytics per la Software Engineering	23
5.1	Definizione di Machine Learning, ruolo di statistica e informatica	23
5.2	Esempi d'uso: associazioni, classificazione, regressione	23
5.2.1	Apprendimento di associazioni	23
5.2.2	Classificazione	24
5.2.3	Regressione	24
5.3	Tipi di apprendimento: supervised, unsupervised, semi-supervised, reinforcement	24
5.3.1	Supervised learning	24
5.3.2	Unsupervised learning	25
5.3.3	Semi-supervised learning	25
5.3.4	Reinforcement learning	25
5.4	Etica del data mining e del ML	25
5.5	Terminologia: istanze, feature, attributi nominali/numerici, output, modelli	26
5.5.1	Instances	26
5.5.2	Features o attributi	26
5.5.3	Output e modelli	26
5.6	Software quality e costo dei bug	27
5.7	Software analytics: definizione, obiettivi e fonti dati	27
5.8	AI/ML applicati alla Software Engineering	28
5.8.1	Applicazioni alla qualità	28
5.8.2	Applicazioni alla produttività	28
5.9	Defect prediction e ML for software defects	29
5.10	Pipeline: measure, clean data, analyze, model, explain	29
5.11	Estrazione dati da ITS e VCS e raccolta metriche	30
5.12	Identificazione difetti, labeling e defect dataset	30
5.13	Black-box models e necessità di spiegazione	31
5.14	XAI, GDPR, fairness, accountability, transparency	31
5.15	CMMI Level 5 e process improvement	32
5.16	Process chart e controllo statistico	32
5.17	Casi applicativi mostrati nelle slide, anche se solo introdotti	33
5.17.1	Simulatore di release e defect prediction	33
5.17.2	Traceability recovery	33
5.17.3	Defectiveness prediction via JIT	34
5.18	Riepilogo finale	34
6	SZZ: Identifying Buggy Entities	35
6.1	Contesto	35
6.1.1	Cos'è la defect prediction	35
6.1.2	Perché la qualità del dataset è fondamentale	35
6.1.3	Principali fonti di rumore nei dataset	35
6.2	Cos'è SZZ	36
6.2.1	Definizione e obiettivo	36
6.2.2	Uso del versioning e dell'annotation	36
6.2.3	Idea di partire dal fix per risalire al bug-introducing change	36
6.3	Come funziona SZZ	37
6.3.1	Identificazione del fix commit	37
6.3.2	Analisi delle linee modificate	37

6.3.3	Ricerca all'indietro dell'ultima modifica precedente	37
6.3.4	Differenza tra bug-fix change e bug-introducing change	37
6.4	Esempi nelle slide	38
6.4.1	Esempio 1: "When Do Changes Induce Fixes?"	38
6.4.2	Esempio 2: "The Impact of Refactoring Changes on the SZZ Algorithm: An Empirical Study"	38
6.5	Limiti di SZZ	38
6.5.1	Code additions	38
6.5.2	Regressive bugs	39
6.5.3	Refactoring	39
6.5.4	Imprecise backward search	39
6.5.5	Snoring	39
6.6	Da ricordare per l'esame	39
7	Leveraging Defects Lifecycle for Labeling Defective Classes	41
7.1	Contesto e obiettivo del lavoro	41
7.1.1	Defect prediction	41
7.1.2	Importanza del dataset	41
7.1.3	Labeling delle classi difettose	41
7.1.4	Obiettivo generale dello studio	41
7.2	Concetti fondamentali	42
7.2.1	SZZ	42
7.2.2	Earliest AV in JIRA	42
7.2.3	Defect lifecycle	42
7.2.4	Significato di IV, OV, FV e AV	42
7.3	SZZ: funzionamento e limiti	42
7.3.1	Come funziona SZZ	42
7.3.2	Uso di <code>git blame</code> / annotation mechanism	43
7.3.3	Bug introducing commit	43
7.3.4	Limiti del metodo	43
7.4	Aim dello studio e Research Questions	43
7.4.1	Idea di stable life-cycle dei defect	43
7.4.2	Research Questions	43
7.5	RQ1: Is the AV available and consistent?	44
7.5.1	Razionale	44
7.5.2	Design e selezione di progetti e bug	44
7.5.3	Risultati	44
7.5.4	Discussione e implicazioni pratiche	44
7.6	RQ2: Do methods have different accuracy in AV labeling?	45
7.6.1	Razionale	45
7.6.2	Formula della Proportion e significato di P	45
7.6.3	Varianti di Proportion	45
7.6.4	Metodi confrontati	46
7.6.5	Metriche	46
7.6.6	Risultati e discussione	46
7.7	RQ3: Do methods have different accuracy in classes labeling?	47
7.7.1	Differenza rispetto a RQ2	47
7.7.2	Design	47

7.7.3	Risultati	47
7.7.4	Discussione	47
7.8	RQ4: Do methods lead to different identification of important features? . .	48
7.8.1	Razionale	48
7.8.2	Feature selection	48
7.8.3	Correlation analysis e Standard Mean Relative Error	48
7.8.4	Risultati	48
7.8.5	Discussione	48
7.9	Figure, tabelle e workflow del paper	48
7.9.1	Diagramma del defect lifecycle	49
7.9.2	Workflow del predicted AV	49
7.9.3	Workflow del class labeling	49
7.9.4	Workflow dei complete datasets	49
7.9.5	Feature selection e correlation analysis	49
7.9.6	Interpretazione di grafici, boxplot e tabelle	49
7.10	Conclusioni e future work	50
7.10.1	Risultati principali del paper	50
7.10.2	Perché AV non basta	50
7.10.3	Perché Proportion è rilevante	50
7.10.4	Significatività statistica vs rilevanza pratica	50
7.10.5	Sviluppi futuri	50
8	Defect Labeling, Evaluation and Metrics	51
8.1	RQ3: Accuratezza nel labeling delle classi	51
8.1.1	Differenza rispetto a RQ2	51
8.1.2	Unità di analisi	51
8.1.3	Design	51
8.1.4	Risultati	51
8.1.5	Confronto tra metodi	52
8.2	RQ4: Identificazione delle feature importanti	52
8.2.1	Razionale	52
8.2.2	Feature selection	52
8.2.3	Correlation analysis	52
8.2.4	Standard Mean Relative Error	52
8.2.5	Risultati	53
8.3	Figure, tabelle e workflow del paper	53
8.3.1	Diagramma del defect lifecycle (Fig. 1.1)	53
8.3.2	Workflow del predicted AV (Fig. 3.1)	53
8.3.3	Workflow del class labeling (Fig. 3.2)	53
8.3.4	Workflow della creazione dei complete datasets (Fig. 3.3)	53
8.3.5	Feature selection e correlation analysis (Fig. 3.4 e Fig. 3.5)	54
8.3.6	Interpretazione di grafici, boxplot e tabelle comparative	54
8.4	Evaluation of Machine Learning Models	54
8.4.1	Training, testing e tuning	54
8.4.2	Perché l'errore sul training set non basta	54
8.4.3	Training set vs test set	54
8.4.4	Dati rappresentativi	54
8.4.5	Problema dei dati limitati	54

8.5	Tecniche di validazione non temporali	55
8.5.1	Holdout	55
8.5.2	Stratification	55
8.5.3	Repeated holdout	55
8.5.4	K-fold cross-validation	55
8.5.5	Stratified k-fold cross-validation e repeated stratified cross-validation	56
8.5.6	Leave-one-out cross-validation	56
8.5.7	Bootstrap	56
8.6	Tecniche di validazione temporali	56
8.6.1	Time-series validation	56
8.6.2	Ordered holdout	56
8.6.3	Walk-forward	56
8.6.4	Importanza dell'ordine temporale	57
8.7	Performance metrics	57
8.7.1	Binary classifier	57
8.7.2	Confusion matrix e metriche base	57
8.7.3	Precision	57
8.7.4	Recall	58
8.7.5	Accuracy	58
8.7.6	Kappa	58
8.7.7	Dipendenza dal contesto applicativo	58
8.8	ROC curves e AUC	58
8.8.1	Threshold dependence	58
8.8.2	ROC curve	58
8.8.3	AUC	59
8.9	Collegamento tra i due PDF	59
8.9.1	Come il labeling influenza la qualità del dataset e del modello . . .	59
8.9.2	Realismo temporale e defect prediction	59
8.9.3	Scegliere bene metriche e validazione	59
8.10	Conclusioni finali	60
8.10.1	Risultati del paper Proportion e limiti dell'approccio Jira-based . .	60
8.10.2	Confronto Proportion vs SZZ	60
8.10.3	Conclusioni su validazione e metriche	60
9	Snoring: rumore nei dataset di defect prediction	61
9.1	Introduzione e contesto della defect prediction	61
9.2	Sleeping defects e snoring classes	61
9.3	Come vengono assegnati i difetti alle classi	62
9.4	SZZ: funzionamento e limiti	62
9.4.1	Funzionamento	62
9.4.2	Limiti	63
9.4.3	Spiegazione del diagramma mostrato nelle slide	63
9.5	Obiettivi dello studio e Research Questions	63
9.6	RQ1: To what extent do defects sleep?	63
9.6.1	Razionale	63
9.6.2	Design	63
9.6.3	Variabili e metriche	64
9.6.4	Risultati	64

9.6.5	Discussione	64
9.7	RQ2: To what extent do classes snore?	64
9.7.1	Razionale	64
9.7.2	Design	64
9.7.3	Variabili e metriche	64
9.7.4	Risultati	65
9.7.5	Discussione	65
9.8	RQ3: To what extent does snoring impact the evaluation of classifiers accuracy?	65
9.8.1	Razionale	65
9.8.2	Design	65
9.8.3	Variabili e metriche	65
9.8.4	Risultati	66
9.8.5	Discussione	66
9.9	RQ4: To what extent does snoring impact defect prediction accuracy? . . .	66
9.9.1	Razionale	66
9.9.2	Design	66
9.9.3	Variabili e metriche	66
9.9.4	Risultati	66
9.9.5	Discussione	67
9.10	RQ5: To what extent is no data better than snoring data?	67
9.10.1	Razionale	67
9.10.2	Design	67
9.10.3	Variabili e metriche	67
9.10.4	Risultati	67
9.10.5	Discussione	67
9.11	Conclusioni	68
9.12	Future works	68
10	Effort-Aware Accuracy Metrics e valutazione dei classificatori	69
10.1	Contesto generale	69
10.1.1	Defect prediction e testing activity context	69
10.1.2	Perché l'accuratezza classica non basta	69
10.1.3	Effort-aware accuracy metrics	69
10.2	Concetti fondamentali	70
10.2.1	Definizione di EAM	70
10.2.2	Ranking delle entità difettose	70
10.2.3	Relazione tra effort e LOC da ispezionare	70
10.2.4	Differenza tra ranking per probabilità e ranking effort-aware	70
10.3	PofBx e NPofBx	70
10.3.1	Definizione di PofBx	70
10.3.2	Definizione di NPofBx	70
10.3.3	Significato della normalizzazione	70
10.3.4	Ranking per predicted probability / size	71
10.3.5	Differenza pratica tra i due approcci	71
10.4	Esempio introduttivo	71
10.4.1	Dataset di esempio	71
10.4.2	Significato di PofB20	71

10.4.3	Significato di NPofB20	71
10.4.4	Perché la normalizzazione migliora il numero di difetti trovati . . .	71
10.5	Aim del lavoro e Research Questions	72
10.5.1	Problemi nell'uso delle EAM	72
10.5.2	Proposta di normalizzazione	72
10.5.3	Research Questions	72
10.6	RQ1	72
10.6.1	Numero di studi, progetti, EAM e classifier	72
10.6.2	Risultati principali	72
10.7	RQ2	73
10.7.1	Intuizione di base	73
10.7.2	Variabili indipendente e dipendente	73
10.7.3	PofB10–PofB90 e average	73
10.7.4	Preprocessing	73
10.7.5	Walk-forward	73
10.7.6	Classifier usati	74
10.7.7	Risultati	74
10.7.8	Relative gain	74
10.7.9	Cliff's delta e p-value	74
10.8	RQ3	74
10.8.1	Spearman rank correlation	74
10.8.2	Confronto ranking con e senza normalizzazione	74
10.8.3	Best classifier	75
10.8.4	Risultati e interpretazione pratica	75
10.9	RQ4	75
10.9.1	Confronto tra diverse NPofBx	75
10.9.2	Ruolo di x da 10 a 90	75
10.9.3	Average NPofB e confronto dei ranking	75
10.9.4	Risultati e best classifier	75
10.10	Figure, tabelle e grafici	75
10.10.1	Esempio PofB20 e NPofB20	76
10.10.2	Tabella delle EAM usate negli studi precedenti	76
10.10.3	Tabella dei relative gains	76
10.10.4	Tabella degli equivalent best classifiers	76
10.11	Main results e conclusioni	76
10.11.1	Superiorità delle NPofBs rispetto alle PofBs	76
10.11.2	Cambiamento del best classifier dopo la normalizzazione	76
10.11.3	Implicazioni pratiche	76
10.12	ACUME tool	77
10.12.1	Scopo	77
10.12.2	Input e output	77
10.12.3	Utilità per i ricercatori	77
11	Snoring: rumore nei dataset di defect prediction	78
11.1	Introduzione e contesto della defect prediction	78
11.2	Sleeping defects e snoring classes	78
11.3	Come vengono assegnati i difetti alle classi	79
11.4	SZZ: funzionamento e limiti	79

11.4.1	Funzionamento	79
11.4.2	Limiti	79
11.4.3	Spiegazione del diagramma SZZ nelle slide	80
11.5	Obiettivi dello studio e Research Questions	80
11.6	RQ1: To what extent do defects sleep?	80
11.6.1	Razionale	80
11.6.2	Design	80
11.6.3	Variabili e metriche	80
11.6.4	Risultati	81
11.6.5	Discussione	81
11.7	RQ2: To what extent do classes snore?	81
11.7.1	Razionale	81
11.7.2	Design	81
11.7.3	Variabili e metriche	81
11.7.4	Risultati	81
11.7.5	Discussione	81
11.8	RQ3: To what extent does snoring impact the evaluation of classifiers accuracy?	82
11.8.1	Razionale	82
11.8.2	Design	82
11.8.3	Variabili e metriche	82
11.8.4	Risultati	82
11.8.5	Discussione	82
11.9	RQ4: To what extent does snoring impact defect prediction accuracy? . . .	83
11.9.1	Razionale	83
11.9.2	Design	83
11.9.3	Variabili e metriche	83
11.9.4	Risultati	83
11.9.5	Discussione	83
11.10	RQ5: To what extent is no data better than snoring data?	83
11.10.1	Razionale	83
11.10.2	Design	84
11.10.3	Variabili e metriche	84
11.10.4	Risultati	84
11.10.5	Discussione	84
11.11	Conclusioni	84
11.12	Future works	85

Capitolo 1

Teoria della Misurazione e Scale di Misura

1.1 Introduzione alla Misurazione del Software

La misurazione nel software engineering non è un fine, ma uno strumento decisionale di supporto. Come ricordano Lord Kelvin e Tom DeMarco, l'assenza di misurazione impedisce il controllo e, di conseguenza, il miglioramento dei processi e dei prodotti[cite: 330, 341].

1.1.1 Definizioni Formali

Secondo i modelli standard, è necessario distinguere tre concetti fondamentali:

- **Misura (Measure):** Rappresentazione numerica di una caratteristica o di un attributo di un prodotto o processo software (es. 5000 LOC, 30 mesi-uomo)[cite: 363, 364, 365, 368].
- **Metrica (Metric):** La scala o il sistema di misurazione utilizzato per quantificare una misura[cite: 352]. Definisce le regole di assegnazione dei valori (es. Complessità Ciclomatica, Densità dei Difetti)[cite: 353, 354].
- **Misurazione (Measurement):** Il processo pratico di assegnazione dei numeri alle misure utilizzando una metrica[cite: 357].

1.1.2 Proprietà di una Metrica di Qualità

Per essere considerata valida, una metrica deve soddisfare i seguenti criteri:

Rilevanza: Deve misurare un attributo significativo e importante del prodotto o processo software[cite: 380].

Oggettività: Il risultato deve basarsi su dati verificabili e riproducibili, non su opinioni o stime soggettive[cite: 381, 382].

Affidabilità e Consistenza: La metrica deve produrre risultati costanti in condizioni diverse e deve essere coerente con gli obiettivi del progetto[cite: 383, 384].

Usabilità: Il calcolo e l'interpretazione devono essere facili, intuitivi e non richiedere sforzi o risorse eccessive[cite: 385, 386].

Nota del Prof

La pertinenza di una metrica è strettamente legata al contesto di business. Un errore critico in un sistema assicurativo potrebbe riguardare l'accuratezza dei calcoli (oracolo) con conseguenze disastrose, mentre per un motore di ricerca come Google la priorità assoluta risiede nella disponibilità del servizio (uptime), dove anche poche ore di downtime globale risultano inaccettabili.

1.2 Tassonomia delle Scale di Misura

La scelta della scala utilizzata dipende dalla natura dei dati misurati e determina quali operazioni statistiche siano matematicamente applicabili[cite: 407, 459, 477].

1.2.1 Scala Nominale e Ordinale

- **Nominale:** Utilizzata per assegnare valori a etichette o categorie senza un ordine intrinseco o magnitudine (es. tipi di difetti: logica, UI, performance, sicurezza)[cite: 417, 418, 419]. Operazioni ammesse: Moda, frequenza[cite: 459].
- **Ordinale:** Prevede un ordine di rango per le categorie, ma la distanza o magnitudine tra i valori non è definita né significativa (es. priorità dei difetti: Critical, High, Medium, Low)[cite: 422, 423, 424]. Operazioni ammesse: Mediana[cite: 459].

Nota del Prof

Le scale ordinali, pur limitando le analisi statistiche alla mediana, offrono spesso una maggiore oggettività rispetto a scale numeriche forzate. Valutare la severità di un bug su una scala 0-100 introduce un'alta soggettività (es. la differenza tra 65 e 70 è arbitraria), mentre una distinzione qualitativa (Blocker vs Non-Blocker) risulta più robusta, netta e oggettiva.

1.2.2 Scale Numeriche (Intervallo e Rapporto)

In queste scale i valori sono assegnati in ordine di rango e la magnitudine della differenza tra i punti ha un significato reale e misurabile[cite: 427, 433].

- **Intervallo:** La magnitudine delle differenze è significativa, ma non possiede uno zero assoluto reale[cite: 427, 428].
- **Rapporto (Ratio):** È il tipo di scala più sofisticato, in cui la magnitudine delle differenze è calcolabile e possiede uno zero assoluto reale (es. LOC, sforzo, tempo)[cite: 433, 434, 435].

Statistiche ammesse per entrambe: Media, Deviazione Standard, Varianza, Range (per la scala di Rapporto sono ammessi anche coefficiente di variazione e media geometrica)[cite: 459].

1.3 Modelli di Qualità: Lo Standard ISO/IEC 25010

Lo standard ISO 25010 definisce una tassonomia dettagliata degli attributi di qualità (qualità del prodotto e qualità in uso)[cite: 33]. Le caratteristiche principali individuate dallo standard sono:

1. **Functional Suitability (Adeguatezza funzionale):** Completezza, correttezza e adeguatezza funzionale[cite: 39, 40, 41, 42].
2. **Performance Efficiency (Efficienza delle prestazioni):** Comportamento temporale, utilizzo delle risorse e capacità strutturale[cite: 43, 44, 45, 46].
3. **Security (Sicurezza):** Riservatezza, integrità, non ripudio, responsabilità e autenticità[cite: 47, 48, 49, 50, 51, 52].
4. **Maintainability (Manutenibilità):** Modularità, riusabilità, analizzabilità, modificabilità e testabilità[cite: 53, 54, 55, 56, 57, 58].

Nota del Prof

È cruciale distinguere sempre il modello di qualità dalle metriche: il modello (come l'ISO 25010) definisce *cosa* significa la qualità fornendone una tassonomia, mentre le metriche definiscono *come* quantificarla operativamente[cite: 30]. L'adozione di questi standard fallisce sistematicamente se manca la "buona fede" aziendale; l'uso delle metriche al solo scopo di ottenere certificazioni formali di facciata, senza mirare a un miglioramento sostanziale del prodotto, svalisce l'utilità dell'intero sistema di valutazione ingegneristico.

1.4 Introduzione alle Metriche Object-Oriented

Le metriche object-oriented vengono impiegate per misurare la qualità dei sistemi software ad oggetti e identificare tempestivamente difetti di progettazione o futuri problemi di manutenibilità[cite: 68, 69]. Vanno sempre interpretate come indicatori di rischio e valutate in modo comparativo (rispetto a una baseline o a un trend) e non in senso assoluto[cite: 73].

Nota del Prof

In ambito manageriale, è molto più efficace adottare un approccio pratico basato sulle distribuzioni piuttosto che sulle medie globali. Risulta strategicamente più utile concentrare le attività di analisi, refactoring e test mirati esclusivamente sul 10% delle classi che presentano i valori peggiori di complessità (*outliers*), piuttosto che tentare di ottimizzare e riscrivere l'intero sistema in modo uniforme.

Capitolo 2

GQM+S, Stima dei Costi e Planning Poker

2.1 GQM+Strategies (Goal-Question-Metric + Strategies)

L'approccio GQM+Strategies (GQM+S) è una metodologia che combina il metodo classico GQM con l'identificazione delle migliori strategie per raggiungere gli obiettivi. Questo approccio collega a cascata gli obiettivi aziendali alla misurazione:

Business Goals → Software Goals → Questions → Measures → Strategies / Targets.

Nota del Prof

Il CQM originale è stato co-creato dal Professor Cantone. L'estensione "+ Strategies" serve ad associare l'azione tecnica al risultato voluto. Tutto parte dal Business: ogni feature o refactoring viene fatto in primis per raggiungere obiettivi aziendali.

2.1.1 Il Reference Process (I 6 Step del GQM+S)

Il processo si sviluppa in sei step principali per permettere il miglioramento continuo:

0. **Initialize:** Impegno (Commitment), Staffing e Responsabilità.
1. **Characterize:** Definire lo scope dell'applicazione e caratterizzare il contesto ambientale.
2. **Set Goals:** Analisi dello status quo, prioritizzazione degli obiettivi e costruzione della griglia goals-strategies.
3. **Choose Process:** Pianificare l'implementazione delle strategie e organizzare la raccolta/analisi dei dati.
4. **Execute Model:** Esecuzione delle strategie, raccolta dati e feedback.
5. **Analyze Results:** Analisi dei dati, revisione delle strategie e analisi costi/benefici.
6. **Package and Improve:** Adattamento dei goal e registrazione delle "lessons learned".

Nota del Prof

L'ultimo step di "packaging" si basa sul concetto di *Experience Factory*: lo scopo è impacchettare l'esperienza in una knowledge base per riutilizzarla in futuro.

Solo nelle slide (non spiegato a voce)

Esempi di utilizzo tipici:

- **Miglioramento del processo:** Ridurre lead time o rework (Metriche: cycle time, difetti sfuggiti).
- **Project management:** Migliorare prevedibilità (Metriche: milestone burndown, throughput).
- **Supporto decisionale:** Valutare l'adozione di nuove tecnologie (Metriche: esiti progetto pilota).

2.2 Introduzione alla Stima dei Costi

La stima si basa su tre domande fondamentali: quanto sforzo (*effort*) è richiesto, quanto tempo di calendario è necessario e qual è il costo totale dell'attività.

Nota del Prof

Bisogna distinguere l'effort (lavoro puramente impiegato) dal tempo di calendario: un'attività può richiedere poco sforzo ma molto tempo se le risorse non sono subito disponibili.

2.2.1 Componenti del Costo ed Overheads

Il costo include Hardware/Software, viaggi, formazione e l'Effort, che è il fattore dominante (salari e costi sociali). Gli **Overheads** includono:

- Costi degli edifici (riscaldamento, luce).
- Costi di rete e comunicazione.
- Strutture condivise come mensa o biblioteca.

Nota del Prof

Il salario dello sviluppatore a volte non copre neanche il 50% del costo totale per un'azienda a causa degli overhead. Inoltre, il software ha un costo marginale di replicazione quasi nullo: vendere a un secondo cliente non costa nulla di più in termini di produzione.

2.2.2 Il Ruolo del Linguaggio di Programmazione

Solo nelle slide (non spiegato a voce)

Il linguaggio impatta drasticamente su tempi e costi. Ad esempio, l'Assembly richiede molto più tempo per coding (8 settimane per 5000 linee) e testing (10 settimane) rispetto a un linguaggio ad alto livello per lo stesso sistema.

2.3 Tecniche di Stima

IMPORTANTE PER L'ESAME

All'esame non vengono chieste le tecniche a memoria, ma i vantaggi, gli svantaggi e quando usare l'una rispetto all'altra.

- **Algorithmic cost modelling:** Approccio basato su formule e dati storici, generalmente basato sulla dimensione del software (es. COCOMO).
- **Expert judgement:** Uno o più esperti usano la loro esperienza per predire i costi.
 - *Vantaggi:* Metodo relativamente economico; accurato se gli esperti hanno esperienza diretta in sistemi simili.
 - *Svantaggi:* Molto inaccurato se mancano veri esperti nel dominio.
- **Estimation by analogy:** Il costo è calcolato comparando il progetto con uno simile nello stesso dominio.
 - *Vantaggi:* Accurato se sono disponibili dati di progetti passati.
 - *Svantaggi:* Impossibile se non si è mai affrontato un progetto comparabile; richiede un database di costi sistematicamente aggiornato.
- **Parkinson's Law:** Il progetto costa qualunque risorsa sia disponibile. Il lavoro si espande per riempire il tempo disponibile.
 - *Vantaggi:* Non si va mai oltre il budget (*no overspend*).
 - *Svantaggi:* Il sistema rimane solitamente incompleto (*unfinished*).
- **Pricing to win:** Il costo è pari a quanto il cliente può spendere.
 - *Vantaggi:* Permette di vincere il contratto.
 - *Svantaggi:* Bassa probabilità che il cliente ottenga ciò che vuole; i costi non riflettono il lavoro reale richiesto.

2.4 Planning Poker

Il Planning Poker è una tecnica Agile per la stima del consenso, basata sulle carte di James Grenning.

2.4.1 Dinamiche e Regole del Gioco

Ogni partecipante riceve un mazzo di carte con una sequenza numerica, solitamente di **Fibonacci** (1, 2, 3, 5, 8, 13, 21, ecc.).

- **La Sfida:** Il moderatore presenta una *user story* e il team può fare domande al Product Owner.
- **Voto Segreto:** Ogni membro seleziona privatamente una carta che rappresenta la "grandezza" (*size*) del task, considerando tempo, rischio e complessità.
- **Svelamento e Dibattito:** Le carte vengono mostrate insieme. Se non c'è consenso, chi ha dato i valori estremi spiega le proprie ragioni. Il gruppo discute e si vota di nuovo finché non si raggiunge un accordo.

IMPORTANTE PER L'ESAME

Perché i "buchi" di Fibonacci? Servono a forzare la granularità corretta. All'aumentare della grandezza aumenta l'incertezza: la sequenza ci spinge a scegliere tra valori distanti (es. 13 o 21), riflettendo la reale imprecisione della stima per task grandi.

Nota del Prof

In caso di mancato consenso, si sceglie la stima superiore per tutelarsi dalla naturale indole ottimista dell'uomo, che tende a sottostimare i tempi ignorando gli imprevisti.

2.4.2 Vantaggi e Consigli Pratici (Tips)

Il Planning Poker favorisce la collaborazione e fa emergere i problemi tecnici in anticipo attraverso il dibattito.

- **Chi vota:** Solo chi farà materialmente il lavoro. I manager non votano per non influenzare le stime al ribasso.
- **Pareggi:** Se il team è diviso tra due taglie consecutive (es. 5 e 8), si sceglie la maggiore e si procede.

Solo nelle slide (non spiegato a voce)

Suggerimenti per l'efficienza:

- Usare un timer per limitare le discussioni a circa un minuto.
- Includere una carta "I need a break" per segnalare la stanchezza.
- Mantenere una **Baseline** consistente tra le varie iterazioni.

Capitolo 3

Introduzione all'Architettura Software

3.1 Cos'è un'Architettura Software

IMPORTANTE PER L'ESAME

L'architettura software è uno dei pilastri dell'ingegneria del software e rappresenta una delle domande più frequenti in sede d'esame.

Secondo lo standard **IEEE 1471**, l'architettura è definita come:

“L'organizzazione fondamentale di un sistema, incarnata nei suoi componenti, nelle loro relazioni reciproche e con l'ambiente, e i principi che ne governano la progettazione e l'evoluzione.”

L'intento principale, come sostenuto da Kruchten, è fornire un **controllo intellettuale** su sistemi di enorme complessità, permettendo agli stakeholder di comprendere la struttura senza annegare nei dettagli implementativi.

3.1.1 Architetture Intended vs Unintended

Ogni sistema possiede intrinsecamente un'architettura, che può essere classificata in due modi:

- **Intended (Pianificata):** Esplicita e documentata prima della scrittura del codice. Segue l'approccio del *Forward Engineering*.
- **Unintended (Accidentale):** Implicita e spontanea. Emerge quando si scrive codice senza un piano; in questo caso, l'architettura deve essere ricostruita a posteriori (*Reverse Engineering*).

Nota del Prof

Immaginate di costruire una casa: nell'approccio *intended* disegnate prima la pianta. Nell'approccio *unintended* costruite le stanze a casaccio e solo alla fine vi accorgete di che forma ha la casa. Ricordate però il Manifesto Agile: l'architettura ha valore solo se serve a produrre codice migliore, non deve essere burocrazia inutile.

3.2 Il Modello 4+1 di Kruchten

Per gestire la complessità, Kruchten propone di decomporre l'architettura in diverse **viste** basate sul principio della *separation of concerns*.

- **Logical View:** Supporta i requisiti funzionali, ovvero ciò che il sistema deve fare per l'utente finale.
- **Development View:** Si concentra sull'organizzazione dei moduli e dei package nel codice sorgente.

Nota del Prof

Un esempio tipico è lo **stack di Android**, che mostra come il sistema sia stratificato dal Kernel Linux fino alle applicazioni.

- **Dynamic View:** Analizza il comportamento a runtime, i processi e le interazioni tra oggetti. Viene solitamente modellata con i *Sequence Diagram*.

Solo nelle slide (non spiegato a voce)

Completamento del modello:

- **Deployment View:** Si occupa della mappatura fisica del software sui nodi hardware e della distribuzione geografica dei server.
- **Scenarios (il “+1”):** Casi d'uso o sequenze di azioni che fanno da collante tra le altre quattro viste, verificandone la coerenza.

Nota del Prof

Pensate a una casa: potete guardare la pianta elettrica, quella idraulica o quella strutturale. Sono tutte viste diverse dello stesso edificio, necessarie a stakeholder diversi (elettricista, idraulico, ingegnere).

3.3 Progettazione e Decision-Making

Il design architeturale si basa su tre fattori: **Riuso** (di pattern o componenti), **Metodo** (tecniche sistematiche) e **Intuizione** (l'esperienza dell'architetto).

3.3.1 L'importanza dei Requisiti Non Funzionali

Nota del Prof

L'architettura è dettata principalmente dai **Requisiti Non Funzionali** (attributi di qualità come sicurezza, velocità, scalabilità). Questi requisiti sono strutturali: non si può “aggiungere la sicurezza” alla fine del progetto. È come una casa: se volete che sia antisismica, deve nascere così. Non potete renderla antisismica una volta costruita senza abbatterla.

3.3.2 Trade-offs e il ruolo dell'Architetto

L'architetto deve bilanciare forze contrastanti provenienti da stakeholder diversi. Le decisioni sono spesso sub-ottimali e prese “al buio” (quando i requisiti sono ancora vaghi).

Solo nelle slide (non spiegato a voce)

Questo dilemma è rappresentato graficamente dalla scelta tra carne “Cotta” o “Cruda”: ogni scelta comporta un vantaggio per uno stakeholder e uno svantaggio per un altro.

3.4 Documentazione: Viste e Punti di Vista

IMPORTANTE PER L'ESAME

Questa è una distinzione fondamentale. Molti studenti confondono i due termini: ricordateli bene per l'esame.

- **Architectural View (Vista):** È la rappresentazione del sistema da una certa prospettiva. È composta da uno o più *modelli* (o diagrammi).
- **Architectural Viewpoint (Punto di Vista):** È l'insieme di regole, convenzioni e linguaggi usati per costruire la vista.

Nota del Prof

Esempio dello stadio: il *Viewpoint* è il concetto teorico di “guardare dalla tribuna stampa” (regole e angolazione). La *View* è la foto specifica che scattate in quello stadio particolare applicando quel punto di vista. In questo corso, Modello e Diagramma sono considerati sinonimi.

3.5 Software Product Lines (SPL)

Una SPL è una famiglia di prodotti che condividono un set comune di funzionalità (**Core Assets**). Si basa sul *Systematic Reuse*.

- **Scoping:** Decidere cosa includere nella piattaforma comune. Se lo scope è troppo largo si spreca soldi; se è troppo stretto si perde l'opportunità di riuso.
- **Vantaggi:** Riduzione drastica dei tempi e dei costi di sviluppo per i prodotti successivi al primo.

Nota del Prof

Pensate alle auto: la Mercedes usa lo stesso tasto per il finestrino su tutti i modelli. La BMW usa lo stesso motore fisico per vari modelli, limitandone la potenza via software. Nel software il riuso è vitale, ma genera conflitti di budget: il Team A spesso non vuole pagare il costo extra per rendere un modulo riutilizzabile anche dal Team B.

3.6 Stili Architetture

Lo **Stile Architetture** è un pattern astratto; l'**Architettura** è la sua applicazione pratica.

1. **Client/Server:** Il server gestisce dati e sicurezza, i client richiedono servizi. Dissaccoppiamento parziale (il server non conosce i client).
2. **Peer-to-Peer:** Ogni nodo è sia client che server.

Nota del Prof

Come BitTorrent: chi scarica deve anche caricare, evitando colli di bottiglia.

3. **Pipes and Filters:** Stream di dati trasformati in sequenza.

Nota del Prof

Usato in sistemi critici come l'avionica. Performance altissime, riuso quasi nullo.

4. **Shared Data:** Comunicazione tramite un database centrale.

Solo nelle slide (non spiegato a voce)

Si divide in **Repository** (l'agente chiede i dati) e **Blackboard** (il sistema notifica gli agenti quando i dati cambiano).

Capitolo 4

Debito Tecnico e Code Smells

4.1 Il Debito Tecnico (Technical Debt)

Il Debito Tecnico (TD) rappresenta l'applicazione di concetti finanziari al dominio dell'ingegneria del software, aggiungendo criteri di tempo ed economia al classico concetto di manutenibilità.

IMPORTANTE PER L'ESAME

Il debito tecnico si scompone in due concetti finanziari fondamentali che lo distinguono dalla semplice manutenibilità:

- **Capitale (Principal):** Il costo (effort) necessario per eliminare il debito, ovvero il tempo speso per il refactoring.
- **Interessi (Interest):** La penalità futura pagata se il debito non viene eliminato, che si traduce in rallentamento dello sviluppo o maggiore probabilità di bug.

4.1.1 Cause e Conseguenze

Il debito emerge tipicamente per ottimizzazioni a breve termine che creano handicap a lungo termine. Le cause includono la **crescita organica** (aumento naturale della complessità) e **scelte opportunistiche** (trade-off per rispettare le scadenze).

Nota del Prof

Spesso si creano trade-off tra qualità interna e "Time to Market". Seguendo il principio Agile "Working code over comprehensive documentation", gli sviluppatori possono trascurare la modularizzazione o i commenti pur di consegnare software funzionante e farsi pagare .

IMPORTANTE PER L'ESAME

Il debito tecnico impatta sempre la release successiva: se il codice non è manutenibile oggi, lo sviluppo futuro sarà più lento e pronò ai difetti.

Solo nelle slide (non spiegato a voce)

Gli analizzatori statici (come SonarQube) faticano a calcolare gli "Interessi" (il costo futuro), rendendoli attualmente una metrica poco chiara.

4.1.2 Strategie di Gestione

Nota del Prof

Il debito tecnico "scade": se un modulo di scarso valore non verrà mai più modificato, non produrrà interessi. È un errore fare refactoring su codice che non verrà più toccato.

Le strategie di gestione includono:

- **Monitoraggio e Refactoring:** Uso di strumenti come SonarCloud per tracciare il trend delle violazioni.
- **Quality Gate:** Una regola che blocca automaticamente i commit contenenti nuove violazioni.

Nota del Prof

Metafora della barca bucata: la prima cosa da fare non è svuotare l'acqua (vecchio debito), ma tappare il buco (usare il quality gate per non farne entrare di nuovo).

4.2 Introduzione ai Code Smells

Occorre distinguere tra tre concetti:

- **Quality Rule:** Principio ingegneristico validato (es. commenti, bassa complessità).
- **Violation o Code Smell:** Porzione di codice non conforme a una regola di qualità.
- **Defect (Bug):** Problema che impatta la qualità esterna (es. crash).

IMPORTANTE PER L'ESAME

I Code Smells **non sono bug**. Non causano malfunzionamenti immediati ma rendono la manutenzione difficile. Agiscono come un "campo minato", aumentando la probabilità che uno sviluppatore introduca difetti reali.

4.3 Catalogo dei "Worst Smells"

I "Worst Smells" sono violazioni che non portano alcun vantaggio in nessuna circostanza, a differenza di alcuni smell "classici" (come God Class o Code Clones) che possono nascere da trade-off temporali.

4.3.1 Esempi di Worst Smells

- **Modificatori non ordinati:** Crea confusione visiva inutile.
- **Object.finalize() sovrascritto:** In Java, forzare la distruzione manuale è spesso deleterio.
- **Codice commentato:** Sporca la codebase; spesso sono rimasugli di prototipi.
- **Mancanza del "break" in uno Switch:** Causa comportamenti imprevisti proseguendo nei case successivi.
- **Concatenazione di Stringhe con '+' in un ciclo:** Degrada gravemente le performance; il refactoring corretto prevede l'uso di **StringBuilder**.
- **Pessimo Naming:** Esempi includono attributi chiamati A, B o C.

Nota del Prof

Chiamare un metodo `ThisIsAGreatMethod1` non fa risparmiare tempo significativo: non siamo in Formula 1 dove si risparmiano millisecondi.

Solo nelle slide (non spiegato a voce)

L'analisi su 27 progetti Apache ha rivelato che i Worst Smells non sono meno frequenti né più severi degli altri, ma non avendo ragioni di esistere, vanno bloccati sempre tramite Quality Gates.

Capitolo 5

Machine Learning e Software Analytics per la Software Engineering

5.1 Definizione di Machine Learning, ruolo di statistica e informatica

Il **Machine Learning (ML)** è lo studio degli algoritmi che migliorano le proprie performance in un determinato **task** attraverso l'**esperienza**. L'obiettivo è ottimizzare un criterio di performance utilizzando dati di esempio o esperienze passate.

Questa disciplina unisce due campi fondamentali:

- **Statistica**: necessaria per trarre inferenze a partire da un campione di dati.
- **Informatica**: necessaria per fornire algoritmi efficienti in grado di risolvere il problema di ottimizzazione, oltre a rappresentare e valutare il modello per l'inferenza.

Nota del Prof

È fondamentale comprendere che ogni misurazione e ogni modello di ML, sebbene si basi su dati raccolti nel passato, ha come unico scopo quello di **prendere decisioni sul futuro**. L'intero sistema si fonda su un assioma filosofico implicito ma cruciale: assumiamo che il futuro sia simile al passato. Senza questa assunzione di base, l'uso dei dati storici per addestrare un modello sarebbe completamente inutile.

5.2 Esempi d'uso: associazioni, classificazione, regressione

Il Machine Learning viene applicato a diverse tipologie di problemi.

5.2.1 Apprendimento di associazioni

L'**apprendimento di associazioni** calcola la probabilità che un evento si verifichi in concomitanza di un altro. Un esempio tipico è la **basket analysis**, in cui si stima la probabilità che chi compra un certo prodotto acquisti anche un altro prodotto.

5.2.2 Classificazione

La **classificazione** serve a dividere i dati in categorie. Tra gli esempi più importanti troviamo:

- **Credit scoring**: differenziare clienti a basso o alto rischio in base a reddito e risparmi.
- **Face recognition**.
- **Character recognition**.
- **Speech recognition**.
- **Diagnosi medica**.
- **Predizione del click** su pubblicità web.

5.2.3 Regressione

La **regressione** viene utilizzata quando l'output non è una categoria, ma un valore numerico continuo. Un esempio classico è la predizione del **prezzo di un'auto usata** in base ai suoi attributi.

Nota del Prof

Sistemi come Google o Netflix basano il loro intero modello di business sulla predizione. Google non vende solo spazi pubblicitari, ma predice quale utente cliccherà su quale banner per massimizzare i profitti. Netflix usa algoritmi di classificazione e raccomandazione per permettere all'utente di trovare rapidamente il contenuto desiderato. Inoltre, i sistemi di autenticazione biometrica non cercano una corrispondenza pixel per pixel, ma utilizzano il ML per capire quali variazioni sono significative e quali no.

Le principali applicazioni del ML si articolano quindi in tre grandi famiglie: **associazioni**, **classificazione** e **regressione**. Esse coprono problemi di co-occorrenza, categorizzazione e stima di valori numerici.

5.3 Tipi di apprendimento: supervised, unsupervised, semi-supervised, reinforcement

I modelli di ML possono apprendere in modi diversi a seconda dei dati disponibili.

5.3.1 Supervised learning

Nel **supervised learning**, il modello riceve sia i dati di addestramento sia gli output desiderati, cioè le **label**.

Nota del Prof

Un esempio è fornire al modello la dimensione di un tumore dicendogli esplicitamente se in passato si è rivelato benigno o maligno, affinché impari a classificarlo in futuro.

5.3.2 Unsupervised learning

Nell'**unsupervised learning**, vengono forniti dati di addestramento senza output desiderati. Lo scopo è trovare struttura, similarità o gruppi latenti nei dati.

Nota del Prof

In questo caso l'obiettivo non è predire un valore esatto, ma organizzare i dati in cluster simili tra loro, ad esempio per raggruppare geni simili o fare network analysis.

5.3.3 Semi-supervised learning

Solo nelle slide (non spiegato a voce)

Il **semi-supervised learning** utilizza dati di addestramento in cui solo una parte limitata delle istanze possiede le label corrette.

5.3.4 Reinforcement learning

Solo nelle slide (non spiegato a voce)

Il **reinforcement learning** è una tipologia di apprendimento basata su un sistema di ricompense e punizioni. Nelle slide è citato, ma non viene approfondito a voce.

Nel complesso, la distinzione tra questi paradigmi dipende dal tipo di supervisione disponibile e dal modo in cui il sistema riceve informazione utile per apprendere.

5.4 Etica del data mining e del ML

L'utilizzo dei dati solleva questioni etiche rilevanti. L'anonimizzazione dei dati è difficile, e l'uso di certe informazioni può condurre a discriminazioni. Per esempio, utilizzare razza, sesso o religione per l'approvazione di un prestito è eticamente problematico, mentre in ambito medico gli stessi dati possono essere legittimi.

Le domande fondamentali sono:

- Chi ha accesso ai dati?
- Per quale scopo sono stati raccolti?
- Quali conclusioni possono essere tratte legittimamente?

Nota del Prof

Quando un sistema ML prende decisioni, la responsabilità è un tema complesso. Se una Tesla a guida autonoma causa un incidente, oggi la responsabilità legale ricade ancora su chi è al volante, anche in virtù degli accordi di licenza accettati dagli utenti. Inoltre, il ML tende a replicare i bias umani presenti nei dati storici: se nel passato una certa etnia o un certo genere ha subito discriminazioni, il modello può apprendere e riprodurre quel comportamento in futuro.

L'etica del ML impone quindi attenzione a **privacy**, **bias**, **discriminazione** e **responsabilità**. I dati storici non sono neutrali, e i modelli possono ereditare le distorsioni del contesto da cui provengono.

5.5 Terminologia: istanze, feature, attributi nominali/numerici, output, modelli

Per utilizzare correttamente gli algoritmi di ML è necessario padroneggiare una terminologia standard.

5.5.1 Instances

Le **instances** sono i singoli esempi indipendenti del concetto da apprendere.

Nota del Prof

In un dataset strutturato a tabella, l'istanza rappresenta la singola riga, ad esempio la combinazione di una specifica classe software in una specifica release.

5.5.2 Features o attributi

Le **feature** sono gli aspetti misurabili di un'istanza. Possono essere:

- **Nominali**: valori simbolici o categorici.
- **Numerici**: valori quantitativi su cui si possono eseguire operazioni matematiche.

Nota del Prof

Le feature rappresentano le colonne della tabella. In genere, l'ultima colonna è l'output o variabile da predire, mentre le altre sono metriche che aiutano nella predizione.

5.5.3 Output e modelli

La conoscenza prodotta da un sistema di ML può essere rappresentata tramite:

- **tabelle decisionali**;
- **modelli lineari**;

- alberi decisionali;
- regole;
- cluster.

IMPORTANTE PER L'ESAME

Molti ambienti di ML, in particolare **Weka**, non supportano correttamente gli attributi con scala ordinale, ad esempio *Low*, *Medium*, *High*. Weka tende a trattarli come semplici categorie nominali, perdendo l'informazione dell'ordine. Per questo motivo, spesso occorre eliminare la colonna oppure trasformarla in una variabile binaria, accettando il relativo trade-off nella perdita di informazione.

Un dataset è dunque una tabella in cui le **righe** rappresentano le istanze e le **colonne** rappresentano le feature. La corretta interpretazione del tipo di attributo è essenziale, soprattutto quando si usano strumenti come Weka.

5.6 Software quality e costo dei bug

La **qualità del software** ha un impatto economico enorme. I bug costano a livello globale migliaia di miliardi di dollari, e le attività di **Quality Assurance (QA)** risultano estremamente costose.

Nei sistemi industriali di grandi dimensioni:

- vengono effettuate migliaia di **code review** al giorno;
- vengono eseguiti milioni di **test**;
- ogni modifica deve essere controllata con attenzione.

Nota del Prof

Poiché il testing è molto oneroso, diventa fondamentale stimare in anticipo quali entità del codice, come classi o metodi, abbiano maggiore probabilità di essere difettose, così da concentrare lì l'effort di QA.

Ne segue che la defect prediction nasce anche come risposta economica e organizzativa al problema della qualità: non è sostenibile controllare tutto con la stessa intensità, quindi occorre **prioritizzare**.

5.7 Software analytics: definizione, obiettivi e fonti dati

La **Software Analytics** è la combinazione di **Software Data** e **Data Analytics**. Il software produce una grande quantità di dati distribuiti tra repository, sistemi di build, log di CI/CD, issue tracker, code review e file di configurazione.

Gli obiettivi principali della Software Analytics sono:

1. **Quality improvement**: prevenire crash e bug.

2. **Process improvement:** capire l'impatto di pratiche come code review e rapid release.
3. **Productivity improvement:** valutare l'effetto della CI e di altre pratiche sul team.
4. **Empirical theory building:** costruire teorie empiriche sull'ingegneria del software.

La Software Analytics sfrutta quindi i dati prodotti dagli strumenti di sviluppo per migliorare **qualità**, **processi**, **produttività** e comprensione empirica del software.

5.8 AI/ML applicati alla Software Engineering

L'Intelligenza Artificiale sta trasformando l'Ingegneria del Software.

5.8.1 Applicazioni alla qualità

Tra le applicazioni principali ci sono:

- **defect prediction;**
- **vulnerability prediction;**
- **malware prediction;**
- **test case generation.**

5.8.2 Applicazioni alla produttività

L'AI può supportare anche la produttività del team, ad esempio tramite:

- generazione di **UI**, requisiti, codice e commenti;
- previsione della **produttività** di sviluppatori o team;
- **recommendation** di developer o reviewer;
- identificazione del **turnover** degli sviluppatori.

Solo nelle slide (non spiegato a voce)

Nelle slide vengono citati esempi industriali concreti, come un **AI coding assistant** sviluppato in collaborazione con Mozilla, i tool **SapFix** e **Sapienz** di Facebook per trovare e correggere bug automaticamente, e **Sketch2Code** di Microsoft per trasformare schizzi su lavagna in codice HTML.

Nel contesto della Software Engineering, l'AI non serve quindi solo a prevedere bug, ma anche a supportare gli sviluppatori nella produzione di codice, nel testing e nella gestione del lavoro di squadra.

5.9 Defect prediction e ML for software defects

Nel contesto dei difetti software, il ML ha tre grandi obiettivi:

1. **Predire** i difetti futuri, per ottimizzare le risorse di QA.
2. **Spiegare** perché un software fallisce.
3. **Costruire** teorie empiriche sulla defectiveness.

Nota del Prof

Prevenire è meglio che curare. Eliminare code smells e individuare le classi più rischiose consente di combattere i bug in modo proattivo e di stabilire priorità di testing più efficaci.

La defect prediction non si limita quindi a indicare *dove* è probabile trovare un difetto, ma cerca anche di chiarire *perché* quel rischio esiste e come ridurlo a livello di processo o di codice.

5.10 Pipeline: measure, clean data, analyze, model, explain

L'applicazione della Software Analytics alla defect prediction segue una pipeline precisa:

1. **Measure**: estrazione dei dati grezzi e raccolta delle metriche.
2. **Clean Data**: pulizia dei dati.
3. **Analyze**: ricerca di correlazioni e relazioni tra feature.
4. **Model**: costruzione e addestramento di modelli analitici o black-box.
5. **Explain**: estrazione di conoscenza utile per spiegare le predizioni.

Nota del Prof

Nel corso verrà seguita esattamente questa pipeline: si partirà da una tabella grezza, la si analizzerà, si sceglieranno e si normalizzeranno le colonne, si costruirà il modello e infine si cercherà di estrarre conoscenza utile a evitare in futuro gli stessi errori.

In sintesi

La pipeline consente di trasformare dati grezzi provenienti dai repository in **modelli predittivi** e poi in **conoscenza spiegabile e azionabile**. È uno schema concettuale fondamentale perché collega raccolta dati, modellazione e interpretazione.

5.11 Estrazione dati da ITS e VCS e raccolta metriche

Il primo passo della pipeline consiste nell'estrazione dei dati da due sorgenti principali:

- **ITS (Issue Tracking System)**: ad esempio Jira, che contiene ticket, bug report e informazioni di progetto.
- **VCS (Version Control System)**: ad esempio Git, che contiene codice sorgente, snapshot e log dei commit.

Da queste fonti si raccolgono diverse famiglie di metriche:

- **Code Metrics**: dimensione, complessità, design object-oriented, accoppiamento, coesione.
- **Process Metrics**: numero di commit, churn, difetti pre-release, pratiche di sviluppo.
- **Human Factors**: code ownership, numero di sviluppatori che hanno toccato il file, esperienza dello sviluppatore.

Nota del Prof

Più persone modificano una stessa classe, maggiore è il rischio di introdurre difetti. Inoltre, se un commit è effettuato da uno sviluppatore che non ha esperienza su quella classe, la probabilità di errore cresce sensibilmente.

Combinando dati da **Jira** e **Git**, si ottengono quindi metriche sul codice, sul processo e sui fattori umani, tutte utili per la defect prediction.

5.12 Identificazione difetti, labeling e defect dataset

Per addestrare un modello di defect prediction è necessario costruire un dataset etichettato, in cui ogni classe venga marcata come **difettosa** o **non difettosa**.

Questo processo dipende dal **linkage** tra:

- ticket nell'Issue Tracking System;
- commit nel Version Control System;
- versioni del software coinvolte nel ciclo di vita del difetto.

Nota del Prof

Quando uno sviluppatore effettua un commit per risolvere un problema, inserisce spesso nel messaggio del commit l'ID del ticket Jira associato. Questo ID rappresenta il razionale del cambiamento. Grazie a questa connessione tra commit e ticket, e tramite tecniche come **SZZ** e **Proportion**, è possibile non solo sapere quale commit ha corretto il bug, ma anche risalire a ritroso fino al commit che lo aveva introdotto.

In sintesi

Il **labeling** dipende dalla capacità di collegare correttamente ticket, commit e release. Questo collegamento è alla base della costruzione del **defect dataset** ed è il passaggio che determina la qualità dei dati su cui verranno addestrati i modelli.

5.13 Black-box models e necessità di spiegazione

Molti algoritmi di apprendimento producono modelli **black-box**: ricevono un input e restituiscono una predizione, ma senza rendere trasparente il ragionamento interno che ha portato a quella decisione.

Per esempio, un modello può dire che una certa classe ha alta probabilità di essere difettosa, ma senza spiegare quali fattori abbiano portato a quel risultato.

Nota del Prof

Un predittore come una rete neurale può essere molto accurato, ma non sa giustificare in modo leggibile il perché della sua scelta. Nell'ingegneria del software, invece, capire il motivo della difettosità è cruciale, perché consente al manager di agire sulla causa e non soltanto sul sintomo.

In contesto ingegneristico, quindi, la sola accuratezza non basta: il valore operativo di una predizione aumenta notevolmente quando essa è accompagnata da una spiegazione comprensibile.

5.14 XAI, GDPR, fairness, accountability, transparency

La necessità di spiegare i modelli è legata sia a motivazioni tecniche sia a motivazioni etiche e legali. In questo contesto si colloca il paradigma **FAT**:

- **Fairness**;
- **Accountability**;
- **Transparency**.

L'**Articolo 22 del GDPR** richiede che i processi decisionali automatizzati che hanno impatto sugli individui siano accompagnati da una spiegazione adeguata.

Questo porta a domande fondamentali:

- I sistemi di AI rispettano le leggi?
- I modelli producono decisioni discriminatorie?
- Comprendiamo davvero perché un modello fa una certa predizione?

Nota del Prof

Una tecnica di Explainable AI consiste nell'affiancare a un modello black-box, come una rete neurale, un modello gemello spiegabile, ad esempio un albero decisionale.

Invece di addestrare l'albero direttamente sui dati originali, lo si addestra sugli output generati dalla rete neurale, così da mimarne il comportamento e renderne più leggibili le decisioni.

In sintesi

La **XAI** nasce dall'esigenza di rendere i modelli comprensibili, verificabili e compatibili con vincoli legali ed etici come quelli imposti dal GDPR. In questo quadro, la spiegabilità non è un'aggiunta opzionale, ma una proprietà sempre più centrale.

5.15 CMMI Level 5 e process improvement

Il **CMMI (Capability Maturity Model Integrated)** è uno standard per misurare la maturità dei processi di sviluppo software.

I livelli principali sono:

1. **Initial**
2. **Managed**
3. **Defined**
4. **Quantitatively Managed**
5. **Optimizing**

Il **Level 5**, cioè *Optimizing*, rappresenta il livello più elevato: le performance del processo vengono migliorate in modo continuo tramite innovazione e controllo quantitativo.

Nota del Prof

Il professore riporta un'esperienza diretta in cui ha aiutato un'azienda a mantenere il Livello 5 del CMMI. L'obiettivo era dimostrare statisticamente che certe strategie di miglioramento avessero prodotto benefici reali. I risultati hanno mostrato una forte riduzione dei difetti e un marcato aumento del profitto per dipendente.

Il riferimento al CMMI mostra bene come metriche, controllo statistico e miglioramento continuo possano essere integrati in una visione strutturata della qualità.

5.16 Process chart e controllo statistico

Per controllare la stabilità del processo si utilizzano i **process chart** o **control chart**. Essi servono a:

- identificare problemi mentre si verificano;
- predire l'intervallo atteso dei risultati del processo;
- verificare se il processo è statisticamente stabile;
- decidere se intervenire con correzioni locali o con cambiamenti più strutturali.

La costruzione di un process chart prevede:

1. scelta della variabile da osservare;
2. scelta dell'unità temporale;
3. calcolo della **media**;
4. calcolo dell'**Upper Control Limit** come media più tre deviazioni standard;
5. calcolo del **Lower Control Limit** come media meno tre deviazioni standard.

Nota del Prof

Finché i dati restano all'interno dei limiti di controllo, il processo è considerato statisticamente stabile. Se invece un punto supera uno dei limiti, si attiva una retrospettiva per analizzare la root cause dell'anomalia e capire come evitare che si ripeta.

I **process chart** permettono dunque di monitorare il processo nel tempo e di individuare rapidamente deviazioni anomale che richiedono analisi e intervento.

5.17 Casi applicativi mostrati nelle slide, anche se solo introdotti

Le lezioni presentano anche alcuni casi applicativi concreti.

5.17.1 Simulatore di release e defect prediction

Nota del Prof

È stata sviluppata un'interfaccia per supportare i manager nel **release planning**. Modificando alcuni slider relativi a variabili di input, come la percentuale di God Classes, la compliance architetturale o il numero di nuovi sviluppatori, il modello ricalcolava dinamicamente il numero atteso di difetti nella release successiva. Il sistema forniva anche intervalli di confidenza e scenari di best case e worst case.

5.17.2 Traceability recovery

Solo nelle slide (non spiegato a voce)

Nelle slide si introducono studi sull'uso di tecniche NLP e classificatori di ML per stimare il numero di link rimanenti nel recupero della tracciabilità dei requisiti.

5.17.3 Defectiveness prediction via JIT

Solo nelle slide (non spiegato a voce)

Le slide finali introducono il tema del miglioramento della defect prediction combinando informazioni **Just-In-Time** a livello di commit con metriche classiche a livello di classe e metodo.

Le tecniche discusse trovano applicazione sia in strumenti concreti di supporto decisionale per i manager, sia in linee di ricerca avanzate su **tracciabilità** e **predizione dei difetti**.

5.18 Riepilogo finale

I concetti chiave emersi sono i seguenti:

- Il **Machine Learning** apprende dai dati passati per prendere decisioni sul futuro.
- Le principali famiglie di problemi sono **associazioni**, **classificazione** e **regressione**.
- La **Software Analytics** usa dati provenienti da repository, issue tracker e pipeline di sviluppo per migliorare qualità, processi e produttività.
- La **defect prediction** serve a stimare quali classi o componenti sono più a rischio di contenere difetti.
- La pipeline fondamentale è: **Measure** → **Clean Data** → **Analyze** → **Model** → **Explain**.
- Il **labeling** delle classi dipende dal corretto collegamento tra ticket, commit e release.
- I modelli **black-box** sono potenti ma poco trasparenti, perciò diventa centrale la **Explainable AI**.
- La spiegabilità è importante non solo dal punto di vista tecnico, ma anche per ragioni di **fairness**, **accountability**, **transparency** e conformità al **GDPR**.
- Il **CMMI Level 5** e i **process chart** mostrano come il controllo quantitativo del processo possa tradursi in miglioramenti concreti di qualità e produttività.

Capitolo 6

SZZ: Identifying Buggy Entities

6.1 Contesto

6.1.1 Cos'è la defect prediction

La **defect prediction** è una tecnica che mira a identificare gli **artefatti software** (come classi, metodi o linee di codice) che hanno un'alta probabilità di presentare un difetto. L'obiettivo principale è **ridurre i costi e i tempi del testing e della code review**, permettendo agli sviluppatori di concentrarsi sulle parti più critiche del codice.

Nota del Prof

Il testing, sia esso automatico o manuale (come l'ispezione del codice), può essere molto oneroso in termini economici e computazionali. Ottimizzare questi sforzi è il vero scopo di queste predizioni. Anche se per semplicità nel progetto si lavora a livello di classe, questi concetti si applicano a vari livelli di granularità, come linea di codice, commit, metodo e classe.

6.1.2 Perché la qualità del dataset è fondamentale

L'**affidabilità di un modello di predizione** dipende direttamente dalla **qualità dei dati** usati per addestrarlo.

Nota del Prof

Vale il principio del *Garbage In, Garbage Out*: se i dati in input contengono impurità o errori, anche l'accuratezza del modello ne risentirà proporzionalmente.

6.1.3 Principali fonti di rumore nei dataset

Le principali **fonti di rumore** indicate sono:

- defect misclassification;
- defect origin.

Solo nelle slide (non spiegato a voce)

Lavori precedenti hanno identificato queste fonti di rumore e hanno proposto delle soluzioni per gestirle.

Nota del Prof

Un esempio pratico di rumore è che circa un terzo dei ticket classificati come “bug” in realtà sono nuove “feature” (*misclassification*). Un'altra fonte di rumore è il **linkage**: se nei commenti dei commit non viene riportato l'ID del ticket (ad esempio di Jira), si perde lo storico delle classi difettose, generando **falsi negativi** nel dataset.

6.2 Cos'è SZZ

6.2.1 Definizione e obiettivo

SZZ è un processo algoritmico usato per identificare il **momento esatto in cui un difetto è stato introdotto** all'interno di un progetto software.

Nota del Prof

Il nome dell'algoritmo deriva semplicemente dai cognomi dei suoi autori.

6.2.2 Uso del versioning e dell'annotation

Il metodo sfrutta il **meccanismo di annotazione** dei sistemi di versionamento, come per esempio il comando `git blame` in Git.

Nota del Prof

L'annotazione permette di analizzare una riga di codice nel suo stato attuale e scoprire in quale revisione precedente è stata modificata l'ultima volta e da chi.

6.2.3 Idea di partire dal fix per risalire al bug-introducing change

L'approccio cerca di determinare, per le specifiche **linee di codice modificate durante la correzione di un difetto**, quando queste erano state alterate per l'ultima volta prima del fix.

IMPORTANTE PER L'ESAME

Il concetto chiave di SZZ è definire come “sorgente” del difetto la stessa esatta zona di codice impattata dalla sua correzione. Se per riparare la macchina il meccanico cambia la trasmissione, il problema era la trasmissione; allo stesso modo, se nel fix di un bug viene modificata una riga di codice, è ragionevole assumere che l'errore risiedesse proprio in quella riga.

6.3 Come funziona SZZ

6.3.1 Identificazione del fix commit

Il primo passo consiste nell'identificare il **fix commit**, cioè il commit che corregge il difetto.

Nota del Prof

Si parte dal sistema di tracciamento, ad esempio Jira, si prendono tutti i ticket chiusi e classificati come "bug". Ogni commit associato a questi ticket rappresenta una **fix** nel codice sorgente.

6.3.2 Analisi delle linee modificate

Una volta trovato il commit di fix, si analizzano le **esatte linee di codice toccate** dalla correzione.

Nota del Prof

Si sa con certezza che in quel momento, prima del fix, in quelle righe era presente un bug.

6.3.3 Ricerca all'indietro dell'ultima modifica precedente

Dopo aver individuato le righe corrette, si usa `git blame` per effettuare una **backward search**, cioè una ricerca all'indietro del commit precedente che aveva inserito o modificato quelle stesse righe.

IMPORTANTE PER L'ESAME

L'entità toccata dal fix era difettosa; SZZ individua la prima modifica precedente a quel fix per etichettare il momento in cui l'errore è nato.

6.3.4 Differenza tra bug-fix change e bug-introducing change

È fondamentale distinguere tra:

- **bug-fix change**: il commit che risolve il problema correggendo il codice;
- **bug-introducing change**: il commit passato che aveva materialmente scritto o alterato in modo errato le linee poi corrette dal fix.

6.4 Esempi nelle slide

6.4.1 Esempio 1: “When Do Changes Induce Fixes?”

Solo nelle slide (non spiegato a voce)

Il primo diagramma mostra una sequenza temporale in cui viene segnalato un bug con ID 42233. Successivamente avviene un fix, nella revisione 1.18, in cui vengono modificati tre metodi: `a()`, `b()` e `c()`. L'algoritmo SZZ traccia all'indietro ciascun metodo per trovare il **bug-introducing change**, scoprendo che `a()` era stato modificato nella revisione 1.11, `b()` nella 1.14 e `c()` nella 1.16.

6.4.2 Esempio 2: “The Impact of Refactoring Changes on the SZZ Algorithm: An Empirical Study”

Nota del Prof

Viene illustrato il caso di un ciclo `for`. Nella versione che introduce il bug (V2), la condizione era scritta come `for(int i=0; i<=4; i++)`. In seguito a un'eccezione riportata nel Bug #123 (V3), il fix altera il ciclo facendolo diventare `for(int i=0; i<4; i++)`.

I diagrammi mostrano operativamente i tre passaggi dell'algoritmo:

1. recuperare il **link nel commit di fix** tramite l'ID del bug;
2. eseguire il **diff** con la versione precedente per isolare la riga corretta;
3. tracciare all'indietro quella specifica riga per individuare il **bug-introducing change**.

Nel caso mostrato, il cambiamento difettoso è la presenza della condizione `i<=4`, poi corretta in `i<4`.

6.5 Limiti di SZZ

Le slide elencano diverse **limitazioni** che riducono l'accuratezza del metodo.

6.5.1 Code additions

Solo nelle slide (non spiegato a voce)

Appena il 26–29% dei **bug-introducing commits** può essere trovato usando l'algoritmo basato sull'assunzione che “un bug è stato introdotto dalle linee di codice che sono state modificate per fixarlo”.

Questo limite riduce l'accuratezza perché, se il fix consiste esclusivamente nell'**aggiunta di nuove righe di codice mancanti**, non esiste una modifica precedente da tracciare all'indietro.

6.5.2 Regressive bugs

Nota del Prof

Sono bug che si creano mentre si sta cercando di implementare qualcos'altro. Questo riduce l'accuratezza perché il codice originario, ad esempio `i<=4`, poteva essere corretto in precedenza e diventare sbagliato solo dopo l'introduzione di un'altra feature. SZZ, facendo la ricerca all'indietro, incolpa ingiustamente lo sviluppatore che aveva scritto quella riga in origine, generando un **falso positivo**.

6.5.3 Refactoring

Nota del Prof

Il **refactoring** è una modifica del codice che ne cambia la struttura ma lascia intatta la funzionalità, ad esempio rinominare una variabile. Se la prima modifica precedente trovata da SZZ è un mero refactoring estetico, l'algoritmo attribuirà erroneamente a quel refactoring l'introduzione del bug, perdendo lo storico reale della funzionalità difettosa.

6.5.4 Imprecise backward search

Nota del Prof

SZZ è molto forte nell'identificare **cosa** è difettoso, ma è impreciso nell'identificare **da quanto tempo** lo è. Un codice poteva essere corretto rispetto a requisiti passati e diventare difettoso solo in seguito a un cambiamento dei requisiti. Se si risale troppo indietro, si etichettano come buggy versioni del codice che, al tempo, erano corrette.

6.5.5 Snoring

Solo nelle slide (non spiegato a voce)

Lo **snoring** è solo nominato nell'elenco puntato della slide senza alcuna descrizione.

Nota del Prof

Il professore lo cita esplicitamente chiarendo che è un concetto legato a come la misurazione del dataset cambia nel tempo, ma rimanda la spiegazione dettagliata alle lezioni successive. In questa lezione, quindi, non viene approfondito.

6.6 Da ricordare per l'esame

- **Definizione di SZZ:** algoritmo che sfrutta l'annotazione dei sistemi di versionamento per identificare il commit in cui un difetto è stato introdotto nel codice sorgente.

- **Passaggi essenziali:** identificare i commit di fix, analizzare le linee modificate da questi fix e ricercare all'indietro, tramite `git blame`, la modifica immediatamente precedente su quelle stesse righe.
- **Differenza tra fix commit e bug-introducing commit:** il primo ripara il difetto, il secondo è la revisione precedente che aveva inserito o alterato in modo errato la riga poi corretta.
- **Principali limiti:** *code additions, regressive bugs, refactoring e imprecise backward search.*

Capitolo 7

Leveraging Defects Lifecycle for Labeling Defective Classes

7.1 Contesto e obiettivo del lavoro

7.1.1 Defect prediction

La **defect prediction** ha l'obiettivo di identificare gli artefatti software, ad esempio classi o file, che hanno un'alta probabilità di manifestare un difetto. Lo scopo principale è ridurre i costi di testing e di code review, permettendo agli sviluppatori di concentrare gli sforzi sulle parti più critiche del codice.

7.1.2 Importanza del dataset

Affinché un modello di predizione sia affidabile, è fondamentale costruire **dataset di difetti di alta qualità**. Modelli addestrati su dati rumorosi raggiungono performance sensibilmente peggiori rispetto a modelli addestrati su dati puliti, in particolare in termini di **Recall**.

7.1.3 Labeling delle classi difettose

Nota del Prof

Ci troviamo nella fase iniziale di **misurazione** e creazione del dataset. Per costruire un modello di Machine Learning, dobbiamo prima etichettare correttamente quali classi siano state effettivamente difettose in passato.

7.1.4 Obiettivo generale dello studio

Lo studio ha tre obiettivi principali:

1. misurare la proporzione di difetti per cui è possibile usare un approccio realistico basato sui dati esatti di Jira;
2. proporre un nuovo metodo automatizzato, chiamato **Proportion**, per recuperare l'origine del difetto quando i dati di Jira mancano;

3. confrontare l'accuratezza del nuovo metodo rispetto alle tradizionali implementazioni di **SZZ**.

7.2 Concetti fondamentali

7.2.1 SZZ

SZZ è l'algoritmo classico per identificare l'origine di un difetto nel codice.

7.2.2 Earliest AV in JIRA

L'approccio **Earliest AV in JIRA** consiste nell'usare la prima **Affected Version** riportata manualmente dagli sviluppatori nel sistema di tracciamento, ad esempio Jira.

7.2.3 Defect lifecycle

Ogni difetto ha un **ciclo di vita** che attraversa diverse versioni o release del software.

7.2.4 Significato di IV, OV, FV e AV

- **IV (Introduction / Injected Version):**

Nota del Prof

è la prima versione del codice in cui il difetto è stato fisicamente scritto o iniettato.

- **OV (Opening Version):** è la versione in cui viene creato il ticket di segnalazione.

Nota del Prof

È il momento in cui la *failure*, cioè il malfunzionamento, viene osservata dall'utente in produzione.

- **FV (Fixed Version):** è la prima versione rilasciata dopo il *fix commit*.

Nota del Prof

È la prima versione del software che non contiene più il bug.

- **AV (Affected Versions):** sono tutte le versioni affette dal difetto, comprese nell'intervallo $[IV, FV)$.

7.3 SZZ: funzionamento e limiti

7.3.1 Come funziona SZZ

SZZ individua i commit che hanno introdotto il bug tracciando la storia del codice sorgente.

7.3.2 Uso di git blame / annotation mechanism

SZZ prende le linee di codice modificate da un **fix commit**, cioè il commit che risolve il bug, e utilizza `git blame` per risalire indietro nel tempo e scoprire quando quelle specifiche linee rimosse o modificate erano state introdotte originariamente.

7.3.3 Bug introducing commit

Nota del Prof

Il commit che ha inserito quella riga originaria viene considerato il **bug introducing commit**, e da quel momento la classe viene etichettata come difettosa.

7.3.4 Limiti del metodo

Le ricerche, e il paper stesso, mostrano che SZZ è lontano dall'essere ideale:

- non è perfettamente accurato;
- non riesce a identificare correttamente i **regression bugs**;
- non funziona quando un difetto viene corretto soltanto con **aggiunta di codice**, senza cancellazione o modifica di righe precedenti, perché non ha linee su cui applicare `git blame`.

Nota del Prof

SZZ genera numerosi **falsi positivi**, quando va troppo indietro nel tempo attribuendo la colpa a linee di codice storicamente corrette, e **falsi negativi**, quando invece non risale abbastanza indietro e non individua il vero punto di introduzione del bug.

7.4 Aim dello studio e Research Questions

7.4.1 Idea di stable life-cycle dei defect

L'intuizione centrale dello studio è che i difetti abbiano un **ciclo di vita stabile**. Esisterebbe cioè una proporzione relativamente costante tra il numero di versioni necessarie per accorgersi di un bug e quelle necessarie per risolverlo.

7.4.2 Research Questions

Le domande di ricerca sono:

- **RQ1:** le **Affected Versions (AV)** sono disponibili e consistenti in Jira?
- **RQ2:** i vari metodi, ad esempio SZZ e Proportion, hanno accuratèzze diverse nell'etichettare le **Affected Versions**?

- **RQ3:** i metodi hanno accuratezze diverse nell'etichettare le **singole classi difettose**?
- **RQ4:** l'uso di metodi diversi porta all'identificazione di **feature importanti** diverse nei modelli di predizione?

7.5 RQ1: Is the AV available and consistent?

7.5.1 Rationale

Si vuole verificare se gli sviluppatori inseriscano regolarmente il campo **Affected Version** in Jira e, nel caso in cui lo facciano, se quel dato sia **consistente**, cioè compatibile con la sequenza temporale delle versioni.

Nota del Prof

Se il bug risulta riportato in una versione futura rispetto all'apertura del ticket, allora il campo è stato compilato in modo errato.

7.5.2 Design e selezione di progetti e bug

Sono stati analizzati **212 progetti Apache** tracciati sia su Git sia su Jira. Sono stati selezionati i ticket che rispettavano i seguenti criteri:

- `Type == "Bug";`
- `Status == "Closed" oppure "Resolved";`
- `Resolution == "Fixed".`

Sono stati esclusi:

- i bug senza un commit di fix collegato su Git;
- i difetti **non post-release**, cioè risolti prima del rilascio in produzione.

7.5.3 Risultati

Su un totale di **125.860 bug validi**, ben **63.539 bug**, cioè il **51%**, non avevano un campo AV utilizzabile oppure presentavano un'AV inconsistente.

7.5.4 Discussione e implicazioni pratiche

Nella maggior parte dei progetti esaminati esiste più del **25%** di difetti senza AV affidabile. Ne segue che l'approccio realistico, basato esclusivamente sulle informazioni di Jira, è **inapplicabile** nella maggioranza dei casi.

Nota del Prof

Se non abbiamo l'AV e non usiamo algoritmi compensativi, perdiamo statisticamente una parte rilevante delle classi difettose, generando molti **falsi negativi**.

7.6 RQ2: Do methods have different accuracy in AV labeling?

7.6.1 Razionale

Dal momento che in RQ1 si osserva che l'AV spesso manca, occorre valutare l'accuratezza di diverse euristiche per calcolarla automaticamente.

7.6.2 Formula della Proportion e significato di P

La costante proporzionale viene definita come:

$$P = \frac{FV - IV}{FV - OV}$$

La formula per predire l'inizio del bug quando manca l'AV è:

$$\text{Predicted IV} = FV - (FV - OV) \cdot P$$

Nota del Prof

L'intuizione è simile a quella delle proporzioni corporee: anche se gli individui hanno altezze diverse, certe relazioni interne tendono a restare stabili. Allo stesso modo, difetti diversi possono avere durate diverse, ma la proporzione tra tempo di scoperta e tempo di correzione tende a mantenersi relativamente costante.

7.6.3 Varianti di Proportion

- **Cold Start:** calcola P come mediana dei valori osservati in tutti gli altri progetti.
- **Increment:** calcola P come media dei difetti corretti nelle versioni precedenti dello stesso progetto.
- **Moving Window:** calcola P come media dell'ultimo 1% dei difetti corretti, risultando più robusto rispetto a outlier e cambiamenti storici di processo.

Nota del Prof

A lezione è stata discussa anche una variante chiamata **Total**, non presente nel paper, che calcola P su tutti i difetti passati e futuri.

IMPORTANTE PER L'ESAME

La variante **Total** è concettualmente sbagliata nel Machine Learning perché viola il **realismo temporale**. Usare dati del futuro per etichettare il passato porta a una sovrastima artificiale delle performance del classificatore. In un contesto reale, il modello non avrebbe mai accesso ai bug futuri per stimare P .

7.6.4 Metodi confrontati

I metodi messi a confronto sono:

- **Simple**, che assume $IV = OV$;
- **SZZ_B** (Basic);
- **SZZ_U** (Unleashed);
- **SZZ_RA** (Refactoring Aware);
- i tre metodi **Proportion**;
- le versioni **SZZ_X+**, che combinano SZZ con i risultati di Simple.

7.6.5 Metriche

Le metriche considerate sono:

- **TP, TN, FP, FN**;
- **Precision**;
- **Recall**;
- **F1**;
- **Kappa**;
- **MCC** (*Matthews Correlation Coefficient*).

IMPORTANTE PER L'ESAME

Il significato pratico di **FP** e **FN** è centrale. Nel contesto software:

- **Falso Negativo (FN)**: il sistema dice che la classe non ha bug, ma in realtà lo ha. È il caso più pericoloso, perché la classe non viene testata e il difetto può arrivare in produzione.
- **Falso Positivo (FP)**: il sistema dice che la classe ha un bug, ma in realtà non ce l'ha. Questo comporta uno spreco di effort di testing, ma non introduce direttamente un bug in produzione.

7.6.6 Risultati e discussione

Dai risultati emerge che:

- tutti i metodi **Proportion** hanno **Precision** e accuratezza composita (**F1, MCC, Kappa**) superiori rispetto a tutti i metodi SZZ;
- **SZZ_U** ottiene la **Recall** più alta in assoluto;
- **SZZ_B+** ha la migliore Precision e le migliori metriche composite all'interno della famiglia SZZ.

Solo nelle slide (non spiegato a voce)

L'incremento di accuratezza ottenuto combinando *SZZ_X* con **Simple**, generando *SZZ_X+*, è statisticamente significativo secondo il *Wilcoxon Signed Rank test* con $p < 0.01$, ma praticamente irrilevante, poiché l'aumento osservato è inferiore all'1%.

7.7 RQ3: Do methods have different accuracy in classes labeling?

7.7.1 Differenza rispetto a RQ2

In RQ2 si misura la validità dell'intervallo temporale delle **Affected Versions**; in RQ3, invece, si passa a una granularità più fine, cioè alla **singola coppia classe-versione**.

7.7.2 Design

Si usano gli stessi metodi e le stesse metriche di RQ2, ma una coppia **classe-release** viene etichettata come difettosa se almeno un **defect-fix commit** ha toccato quella classe in quella specifica versione.

7.7.3 Risultati

I risultati mostrano che:

- anche a livello class-level i metodi **Proportion** ottengono **Precision** e metriche composite migliori rispetto a *SZZ*;
- *SZZ_U* mantiene la **Recall** più alta;
- **Proportion_MovingWindow** domina nettamente tutti i metodi su tutte le metriche composite.

7.7.4 Discussione

Tutti i metodi risultano più accurati nel predire le classi rispetto alle AV. Questo accade perché una singola classe può essere affetta da più difetti simultaneamente: anche se un metodo manca un difetto, può comunque azzeccare l'etichetta grazie a un altro difetto concomitante.

L'incremento mediano osservato passando da AV labeling a class labeling è:

- +13% in **Precision**;
- +16% in **F1**;
- +39% in **Kappa**.

7.8 RQ4: Do methods lead to different identification of important features?

7.8.1 Razionale

RQ4 valuta l'effetto a cascata del labeling. Se il dataset viene etichettato male, allora anche le **feature** selezionate per addestrare il modello potrebbero risultare sbagliate, così come le loro correlazioni con la defectiveness.

7.8.2 Feature selection

Viene usato un approccio di **Exhaustive Search Feature Selection**, tramite Weka e `CfsSubsetEval`, che valuta i sottoinsiemi di feature privilegiando quelle altamente correlate con il difetto e poco inter-correlate tra loro.

7.8.3 Correlation analysis e Standard Mean Relative Error

Per valutare la bontà delle correlazioni si misura la discrepanza tra:

- la correlazione osservata nel dataset **vero**;
- la correlazione osservata nel dataset **stimato** dai vari metodi.

L'errore è definito come:

$$\text{Errore} = \frac{|\text{Predicted} - \text{Actual}|}{\text{Actual}}$$

7.8.4 Risultati

I metodi **Proportion** risultano più accurati dei metodi SZZ in tutte e cinque le metriche considerate. In particolare, mostrano **Precision** e **Recall** mediane perfette, pari al 100%. L'errore dei metodi Proportion è circa il **25%** dell'errore commesso da SZZ.

7.8.5 Discussione

- A differenza di quanto visto nelle altre RQ, esiste poca variazione interna tra le varie implementazioni di SZZ.
- La qualità del labeling influenza direttamente la capacità di identificare le metriche software davvero rilevanti.

Solo nelle slide (non spiegato a voce)

Anche il metodo basilare **Simple** risulta più accurato rispetto a tutti i metodi SZZ in questo task specifico, arrivando a dimezzarne gli errori.

7.9 Figure, tabelle e workflow del paper

Questo capitolo descrive formalmente i workflow visuali presentati nelle slide e nel paper.

7.9.1 Diagramma del defect lifecycle

La figura sul **defect lifecycle** mostra la linea temporale che va da **IV** a **FV**, evidenziando il momento della **Ticket Creation** corrispondente a **OV** e il **Fix Commit** successivo.

7.9.2 Workflow del predicted AV

Il workflow relativo al **predicted AV** descrive il processo seguito in RQ2: i **Bug ID** collegano Jira, da cui si ricava la creazione del ticket, e Git, da cui si ricava il **Fix Hash**. A partire da queste informazioni, il metodo predice una AV che viene poi confrontata con l'**Actual AV**, usata come ground truth.

7.9.3 Workflow del class labeling

Il workflow del **class labeling**, utilizzato in RQ3, associa i file Java o Python toccati dai bug alle rispettive release per verificare i casi di **TP**, **TN**, **FP** e **FN**.

7.9.4 Workflow dei complete datasets

Il processo di creazione dei **complete datasets** estrae dal codice una serie di metriche, tra cui:

- **Size (LOC)**;
- **LOC Touched**;
- **Nfix**;
- **Churn**;
- **Age**;
- **Change Set Size**.

7.9.5 Feature selection e correlation analysis

I workflow successivi mostrano:

- il processo di **feature selection** tramite *Exhaustive Search*;
- l'applicazione della **correlazione di Spearman** tra la feature, ad esempio LOC, e la probabilità di difetto;
- la produzione finale del grafico del **Mean Relative Error**.

7.9.6 Interpretazione di grafici, boxplot e tabelle

I **boxplot** e le **tabelle comparative** confermano visivamente la superiorità strutturale dei metodi **Proportion**, in particolare **Proportion_MovingWindow** e **Proportion_Increment**, rispetto a SZZ in metriche come **MCC** e **Kappa**. Inoltre, la tabella sulla deviazione standard mostra che la deviazione standard di **P** è nettamente inferiore rispetto a quella di **IV**, **OV** e **FV**, rafforzando l'ipotesi di stabilità del ciclo di vita dei difetti.

7.10 Conclusioni e future work

7.10.1 Risultati principali del paper

I risultati principali sono:

1. l'**Affected Version** non può essere usata nel **51%** dei difetti;
2. il metodo **Proportion** è mediamente più accurato di SZZ per etichettare le classi difettose.

7.10.2 Perché AV non basta

Molto spesso gli sviluppatori non compilano il campo AV, oppure lo compilano in maniera inconsistente rispetto alle date di rilascio.

7.10.3 Perché Proportion è rilevante

L'ipotesi che i bug abbiano un **life-cycle stabile** risulta supportata dai dati: la deviazione standard di P all'interno dello stesso progetto è molto bassa. Proportion non solo supera SZZ nel labeling, ma commette anche una frazione molto più piccola dell'errore nell'identificare le feature ingegneristiche rilevanti.

7.10.4 Significatività statistica vs rilevanza pratica

Combinare le informazioni sul ciclo di vita dei defect con i metodi SZZ tradizionali produce un miglioramento **statisticamente significativo**, ma **praticamente trascurabile**, inferiore all'1%.

7.10.5 Sviluppi futuri

Le linee future di ricerca includono:

- analizzare commit bug-introducing successivi in SZZ;
- studiare l'accuratezza delle Affected Versions etichettate dagli sviluppatori Jira;
- replicare l'esperimento nel contesto della **Just-In-Time defect prediction**;
- testare combinazioni più raffinate tra metodi **Proportion** e **SZZ**.

In sintesi

Lo studio dimostra che i dataset di difettosità costruiti esclusivamente con `git blame`, quindi con SZZ, risultano inaccurati. Sfruttare invece una **costante di proporzionalità temporale** del ciclo di vita dei difetti, come avviene nei metodi **Proportion**, permette di costruire dataset più precisi e di migliorare sensibilmente la capacità dei modelli di Machine Learning di individuare i veri predittori dei bug nel codice.

Capitolo 8

Defect Labeling, Evaluation and Metrics

8.1 RQ3: Accuratezza nel labeling delle classi

8.1.1 Differenza rispetto a RQ2

Mentre in RQ2 l'obiettivo era valutare l'accuratezza nell'individuare l'intervallo temporale generale, cioè le **Affected Versions (AV)**, in RQ3 si valuta un livello più granulare per fornire il dataset più accurato possibile per la defect prediction.

8.1.2 Unità di analisi

L'unità di misura non è più la singola versione, ma la coppia **classe-versione**, per esempio *F1.java nella release 1*.

8.1.3 Design

Le variabili indipendenti rimangono i metodi di labeling, cioè **SZZ**, **Proportion** e **Simple**. Per la variabile dipendente, una specifica classe in una specifica versione viene etichettata come difettosa se è stata toccata dal **fix commit** di almeno un difetto in quella versione.

8.1.4 Risultati

I risultati mostrano che:

- come in RQ2, i metodi **Proportion** hanno **Precision** e accuratezza composita, cioè **F1**, **MCC** e **Kappa**, superiori a tutti i metodi **SZZ**;
- **SZZ_U** detiene la **Recall** più alta;
- **SZZ_B+** ha la Precision e l'accuratezza composita più alta all'interno della famiglia **SZZ**;
- a differenza di RQ2, in questa ricerca il metodo **Proportion_MovingWindow** domina in assoluto tutti gli altri metodi sulle metriche composite.

8.1.5 Confronto tra metodi

Solo nelle slide (non spiegato a voce)

Tutti i metodi risultano più accurati nel fare il labeling delle classi rispetto al labeling delle AV. Si osserva un incremento mediano, su metodi e dataset, del 13% in Precision, 5% in Recall, 16% in F1, 27% in MCC e 39% in Kappa. La spiegazione è che una classe può essere affetta da molteplici difetti simultaneamente; quindi, se l'algoritmo manca un difetto, può comunque etichettare correttamente la classe come difettosa grazie alla presenza di un secondo difetto.

Nota del Prof

La Recall di SZZ è più alta perché SZZ va molto più indietro nel tempo rispetto a Proportion, trovando più classi positive, ma generando inevitabilmente anche moltissimi falsi positivi.

8.2 RQ4: Identificazione delle feature importanti

8.2.1 Razionale

Comprendere le metriche, cioè le **feature**, più importanti è vitale per gli sviluppatori, perché consente di evitare condizioni che favoriscono l'introduzione di difetti, come classi troppo grandi o con troppo *churn*. Questa ricerca è duplice: verifica

1. quali feature vengono selezionate dal classificatore;
2. se queste feature sono altamente correlate con la variabile predetta, cioè la difettosità.

8.2.2 Feature selection

Viene utilizzato un approccio chiamato **Exhaustive Search Feature Selection**, tramite Weka e `CfsSubsetEval`. Questo approccio cerca, tra feature come **LOC**, **Churn**, **Age**, **ChangeSetSize** e altre, quelle con alta capacità predittiva individuale e bassa inter-correlazione, così da ridurre la ridondanza.

8.2.3 Correlation analysis

Si confrontano i risultati del dataset generato dai vari metodi con i risultati del dataset **Actual**, calcolando la **correlazione di Spearman**.

8.2.4 Standard Mean Relative Error

Per valutare l'errore si usa la formula:

$$\frac{|\text{Predicted} - \text{Actual}|}{\text{Actual}}$$

8.2.5 Risultati

I risultati mostrano che:

- i metodi **Proportion** hanno un'accuratezza superiore a tutti i metodi SZZ in **tutte e cinque le metriche**;
- i metodi Proportion registrano una mediana perfetta, pari al 100%, sia in **Precision** sia in **Recall**.

Solo nelle slide (non spiegato a voce)

L'errore commesso dai metodi Proportion in questa task è circa il 25% dell'errore commesso dai metodi SZZ. Inoltre, a differenza di RQ2 e RQ3, per la feature selection persino il banale metodo **Simple** è più accurato di SZZ, commettendo circa la metà dell'errore di SZZ. Vi è meno dell'1% di miglioramento usando le varianti SZZ_X+.

8.3 Figure, tabelle e workflow del paper

Solo nelle slide (non spiegato a voce)

Le figure, le tabelle e i workflow descritti in questa sezione sono ricavati dalle slide e dal paper, ma non sono stati spiegati a voce in modo esteso.

8.3.1 Diagramma del defect lifecycle (Fig. 1.1)

Mostra la linea temporale lineare di un bug:

Introduction Version (IV) → Ticket Creation (OV) → Fix Commit → Fixed Version (FV)

Le **AV** si trovano nell'intervallo $[IV, OV]$.

8.3.2 Workflow del predicted AV (Fig. 3.1)

Il workflow mostra come i **Bug ID** uniscano le informazioni di Git, da cui si ricava il **Bug Fix**, e di Jira, da cui si ottengono **Ticket Creation** e **Releases**. Il **Method**, ad esempio Proportion, calcola la **Predicted AV** e la confronta con l'**Actual AV** per ricavare **TP**, **FP**, **TN** e **FN**.

8.3.3 Workflow del class labeling (Fig. 3.2)

Questo workflow prende i file toccati dal **fix** e li unisce alle AV predette, indicando per ogni release se il singolo file fosse difettoso oppure no.

8.3.4 Workflow della creazione dei complete datasets (Fig. 3.3)

Qui viene aggiunta la fase di **Metric Collection** al class labeling. Si ottengono così le tabelle finali contenenti informazioni come **Release**, **File**, **LOC** e altre metriche software.

8.3.5 Feature selection e correlation analysis (Fig. 3.4 e Fig. 3.5)

I dataset, sia quelli predetti sia quelli actual, vengono sottoposti allo step di **Exhaustive Search Feature Selection** oppure al calcolo della **correlazione di Spearman**, per poi unire i risultati nel calcolo del **Relative Error**.

8.3.6 Interpretazione di grafici, boxplot e tabelle comparative

La **Fig. 4.4** e la **Tabella 4.1** mostrano che la deviazione standard di P è molto compatta, circa 5 tra progetti diversi e minore di 2 all'interno dello stesso progetto, a differenza della grande varianza di IV , OV e FV , che supera 38. I boxplot di accuratezza mostrano invece che i metodi SZZ hanno valori molto bassi e schiacciati per **MCC** e **Kappa**, mentre i metodi Proportion risultano più stabili e più alti.

8.4 Evaluation of Machine Learning Models

8.4.1 Training, testing e tuning

Queste sono le fasi critiche per valutare ciò che è stato appreso dal modello. Il **training set** serve a creare il modello, mentre il **testing set** serve a valutarlo su istanze indipendenti.

8.4.2 Perché l'errore sul training set non basta

L'errore misurato sul training set non è un buon indicatore delle performance su dati futuri, perché il modello viene valutato sugli stessi dati con cui è stato costruito.

8.4.3 Training set vs test set

Nota del Prof

Tutti i dati a nostra disposizione, cioè la tabella, sono di tipo *supervised*, quindi conosciamo già l'esito corretto. Per simulare la realtà, nascondiamo i risultati di una parte dei dati, il testing set, per interrogare il modello addestrato sul training set e confrontare poi *actual* e *predicted*.

8.4.4 Dati rappresentativi

Si assume che sia il set di training sia quello di testing siano campioni rappresentativi del problema sottostante. Se il modello deve predire i clienti della città B, allora è sensato testarlo su dati della città B, non della città A.

8.4.5 Problema dei dati limitati

Idealmente entrambi i set dovrebbero essere molto grandi, così da garantire sia addestramento efficace sia stime di errore affidabili. In pratica, però, i dati etichettati sono spesso limitati, e da qui nasce la necessità di tecniche di validazione più sofisticate.

Nota del Prof

È fondamentale ricordare l'assioma di base: la validazione ha senso solo perché assumiamo che **il futuro sia simile al passato**. Misuriamo il passato per stimare le performance su dati futuri che non abbiamo ancora.

8.5 Tecniche di validazione non temporali

Queste tecniche si usano quando l'ordine temporale dei dati non è rilevante oppure quando le istanze sono considerate indipendenti.

8.5.1 Holdout

Il metodo **holdout** divide il dataset originario, per esempio in 2/3 training e 1/3 testing. Il suo svantaggio principale è che il campione potrebbe non essere rappresentativo oppure potrebbe contenere classi assenti nel test.

Nota del Prof

Se una classe rara finisce tutta nel test set, il modello non l'ha mai vista in training e fallirà nel riconoscerla.

8.5.2 Stratification

La **stratification** è una variante dell'holdout che assicura che in ogni subset vi sia approssimativamente la stessa proporzione della classe di interesse.

8.5.3 Repeated holdout

Il **repeated holdout** ripete l'holdout con campionamenti diversi, così da mitigare il problema della non rappresentatività. Gli errori vengono mediati tra le varie esecuzioni. Il limite è che i test set si sovrappongono.

Nota del Prof

Se prima ci mettevo 5 ore, con 5 run ce ne metto 25.

8.5.4 K-fold cross-validation

La **k-fold cross-validation** evita la sovrapposizione dei test set. Divide i dati in K subset uguali; per K volte, un subset fa da test e i restanti da training.

Nota del Prof

L'analogia è quella della pizza: si taglia in 6 spicchi. Se ne toglie uno per testare e si usano gli altri 5 per il training. Poi si rimette lo spicchio, se ne toglie un altro e si ripete per 6 volte. Così ogni dato finisce nel testing set esattamente una volta.

8.5.5 Stratified k-fold cross-validation e repeated stratified cross-validation

L'uso standard è il **10 times 10-fold stratified**. Questa tecnica garantisce stime molto accurate e bassa varianza, ma richiede 100 run per classificatore.

8.5.6 Leave-one-out cross-validation

La **leave-one-out cross-validation** è il caso estremo in cui K è uguale al numero totale di istanze, cioè n . I vantaggi sono il massimo utilizzo dei dati e l'assenza di sottocampionamento casuale. Gli svantaggi sono che la stratificazione è impossibile, dato che il test set contiene una sola istanza, e che il costo computazionale è estremamente elevato.

8.5.7 Bootstrap

Il **bootstrap** usa un campionamento *with replacement*, cioè con reinserimento.

Nota del Prof

Questo metodo mischia troppo le cose. Io personalmente lo sconsiglio. Estrahendo a caso con reinserimento, alcuni dati peseranno moltissimo nel training set e altri non verranno mai visti se non nel test, creando un campione falsato. Non va mai usato nel nostro contesto.

8.6 Tecniche di validazione temporali

Queste tecniche si usano quando i dati sono **time-sensitive**, cioè quando l'ordine temporale è un fattore critico.

8.6.1 Time-series validation

La caratteristica di base è preservare l'ordine temporale dei dati, impedendo che il test set contenga dati antecedenti al training set.

8.6.2 Ordered holdout

L'**ordered holdout** è simile all'holdout, ma lo split, per esempio 80% – 20%, rispetta rigorosamente la cronologia: la parte iniziale va nel training set e la parte finale nel test set.

8.6.3 Walk-forward

Il **walk-forward** è la tecnica più usata. Il dataset viene diviso in parti cronologiche, per esempio le release. A ogni iterazione, tutti i dati disponibili prima della parte da predire formano il training set.

Nota del Prof

Se devo predire la Release 4, il mio training set sarà l'unione delle release 1, 2 e 3.

8.6.4 Importanza dell'ordine temporale

Nota del Prof

Usare dati futuri viola il principio del realismo del machine learning e genera *Garbage In, Garbage Out*. L'approccio di labeling chiamato **Total**, che usa tutto lo storico dei bug, passati e futuri, dà un vantaggio illusorio al modello, perché in uno scenario reale non possiedi le informazioni dei ticket che verranno aperti tra due anni.

8.7 Performance metrics

8.7.1 Binary classifier

Un **binary classifier** è un modello che produce un output con due etichette, ad esempio *Yes/No*, *1/0*, *Positive/Negative*, *Buggy/Non Buggy*.

8.7.2 Confusion matrix e metriche base

- **TP (True Positive)**: predetto positivo, attualmente positivo;
- **FP (False Positive)**: predetto positivo, attualmente negativo;
- **TN (True Negative)**: predetto negativo, attualmente negativo;
- **FN (False Negative)**: predetto negativo, attualmente positivo.

Nota del Prof

Un False Positive significa sprecare tempo, cioè effort, a testare una classe che in realtà è sana.

Nota del Prof

Un False Negative è un errore disastroso: non testo una classe bacata e il bug finisce dritto in produzione in mano all'utente.

8.7.3 Precision

La **Precision** è:

$$\frac{TP}{TP + FP}$$

Nota del Prof

È la favola di “al lupo al lupo”. Se dici sempre che c'è un bug, la tua precisione crolla perché produci tantissimi falsi positivi, cioè fai scattare l'allarme a vuoto.

8.7.4 Recall

La **Recall** è:

$$\frac{TP}{TP + FN}$$

e misura quanti dei positivi reali sei stato in grado di classificare correttamente.

8.7.5 Accuracy

L'**Accuracy** è:

$$\frac{TP + TN}{TP + TN + FP + FN}$$

8.7.6 Kappa

Il **Kappa** misura quanto il classificatore sia più accurato rispetto a un classificatore **Dummy**, cioè casuale o maggioritario. Varia da -1 a 1 , dove 1 indica perfezione, 0 indica una performance equivalente al caso e valori negativi indicano performance peggiori del caso.

Nota del Prof

Un valore di Kappa maggiore di 0.4 potrebbe validare la teoria dell'ingegneria del software nei vostri progetti, indicando che esiste davvero una correlazione tra metriche e difetti.

8.7.7 Dipendenza dal contesto applicativo

Solo nelle slide (non spiegato a voce)

Una Recall alta può essere facilissima da ottenere se il dataset è composto al 99% da istanze positive. Per questo motivo le metriche devono sempre essere interpretate congiuntamente e nel contesto corretto.

8.8 ROC curves e AUC

8.8.1 Threshold dependence

Precision, Recall e F1 dipendono fortemente dalla soglia, cioè dal **threshold**, usato per trasformare la probabilità in una decisione positiva o negativa.

8.8.2 ROC curve

La **ROC curve** si ottiene plottando il **True Positive Rate (TPR)** sull'asse Y contro il **False Positive Rate (FPR)** sull'asse X , al variare di tutte le possibili soglie.

8.8.3 AUC

L'**AUC**, cioè l'area sotto la curva ROC, misura la probabilità che un'istanza positiva scelta a caso venga classificata con probabilità maggiore rispetto a un'istanza negativa scelta a caso.

IMPORTANTE PER L'ESAME

Il professore evidenzia un limite critico dell'AUC: calcolando l'area su tutte le soglie, da 0 a 1, si finisce per mediare anche valori di soglia totalmente irrilevanti per il contesto applicativo. Se il dominio aziendale preferisce i falsi positivi ai falsi negativi, interessano solo certe soglie basse. Mediare anche soglie irrealistiche annacqua il significato reale della metrica.

8.9 Collegamento tra i due PDF

I due argomenti, cioè **labeling dei difetti** e **evaluation dei modelli di Machine Learning**, sono strettamente sequenziali.

8.9.1 Come il labeling influenza la qualità del dataset e del modello

Prima di valutare un modello tramite tecniche come la cross-validation, occorre costruire un **training set** etichettato con una ground truth affidabile. Se per l'etichettatura si usano metodi inaccurati come SZZ, si introducono nel dataset alti livelli di **False Positive** e **False Negative**.

Nota del Prof

L'approccio SZZ va troppo indietro nel tempo, sporcando le release vecchie. Quando validiamo il modello, il classificatore imparerà pattern errati, secondo il principio *Garbage In, Garbage Out*, e fallirà irrimediabilmente la predizione sul test set.

8.9.2 Realismo temporale e defect prediction

Usare **Proportion Increment**, che si basa sui difetti passati, o usare **Walk-Forward**, che addestra solo sulle release precedenti, risponde allo stesso principio etico e matematico: il modello non può barare leggendo il futuro.

8.9.3 Scegliere bene metriche e validazione

Se l'obiettivo aziendale è individuare la classe responsabile dei difetti in produzione per risparmiare risorse, un metodo Proportion garantisce una feature selection più accurata, permettendo al modello di individuare le reali cause del problema, da valutare poi con metriche robuste e coerenti con il contesto, come F1 e Kappa.

8.10 Conclusioni finali

I risultati del paper Proportion e i concetti sulle metriche di valutazione convergono verso una conclusione metodologica comune: il Machine Learning applicato all’Ingegneria del Software richiede **rigore** sia nella costruzione del dataset sia nella valutazione del modello.

8.10.1 Risultati del paper Proportion e limiti dell’approccio Jira-based

I risultati su circa **125.000 difetti** dimostrano che affidarsi esclusivamente ai dati inseriti dagli sviluppatori in Jira è impossibile nel **51%** dei casi, a causa di AV mancanti o incoerenti.

8.10.2 Confronto Proportion vs SZZ

Su circa **6 milioni di classi**, sfruttare la stabilità proporzionale del ciclo di vita dei difetti produce risultati mediamente e significativamente più accurati in **Precision, Kappa, F1** e **MCC** rispetto a SZZ. SZZ resta più alto in Recall, ma questo dipende dal fatto che etichetta come positive molte più classi, pagando però un forte prezzo in Precision.

8.10.3 Conclusioni su validazione e metriche

Nessuna singola metrica è autosufficiente. Per valutare se un modello sia pronto a supportare decisioni reali sulle release, è vitale interpretare più metriche insieme, prestare attenzione alle soglie, e usare tecniche di validazione temporali rigorose, come il **walk-forward**, quando il dataset è legato alla storia evolutiva del software.

Capitolo 9

Snoring: rumore nei dataset di defect prediction

9.1 Introduzione e contesto della defect prediction

La **defect prediction** è una tecnica che mira a identificare gli artefatti software, come le classi, che hanno un'alta probabilità di presentare un difetto.

- **Obiettivo:** ridurre i costi e il tempo dedicati alle fasi di testing e code review, permettendo agli sviluppatori di concentrare gli sforzi sulle parti più critiche del codice.
- **Contesto:** lo studio si colloca nell'ambito della **within-project defect prediction**, cioè la classificazione delle classi difettose all'interno di una specifica release di un progetto.
- **Problema dei dati:** l'affidabilità di un modello predittivo dipende interamente dalla qualità del dataset su cui viene addestrato.

Nota del Prof

Questo richiama il principio del **Garbage In, Garbage Out**: se i dati in ingresso sono inaccurati, i risultati del classificatore saranno inevitabilmente inaffidabili.

Solo nelle slide (non spiegato a voce)

Lavori precedenti hanno già identificato altre fonti di rumore nei dataset, come la **misclassification** e l'origine dei difetti, proponendo soluzioni mirate.

9.2 Sleeping defects e snoring classes

Gli **sleeping defects** sono difetti che vengono scoperti o corretti solo diverse release dopo la loro introduzione. In altre parole, il difetto resta presente nel codice per un certo intervallo di tempo prima di emergere esplicitamente. Nelle slide questo fenomeno è associato ai **post-release defects**.

Le **snoring classes** sono invece classi affette *esclusivamente* da difetti non ancora corretti. Poiché l'esistenza di un difetto non può essere conosciuta prima del suo fix,

queste classi vengono erroneamente considerate pulite. Di conseguenza, esse introducono **rumore** nei dataset di defect prediction.

Nota del Prof

La metafora è semplice: il difetto *dorme*, ma è la classe che *rusa*, cioè genera disturbo nella misurazione. Se una classe contiene più bug dormienti ma anche un solo bug già noto e corretto, allora viene comunque etichettata come difettosa e non rientra nel fenomeno dello snoring. La classe *rusa* solo quando tutti i bug che la colpiscono sono ancora dormienti.

IMPORTANTE PER L'ESAME

Lo **snoring** introduce nei dataset **solo Falsi Negativi (FN)** e mai Falsi Positivi. Una classe etichettata come difettosa lo è con certezza, perché esiste già un fix che la collega a un difetto. Al contrario, una classe etichettata come non difettosa potrebbe in realtà nascondere un difetto ancora dormiente. Per questo motivo, l'ultima release di un progetto appare sempre artificialmente migliore: i bug possono già esserci, ma non sono ancora stati scoperti.

9.3 Come vengono assegnati i difetti alle classi

Per stabilire da quando una classe è difettosa, il processo segue due strade principali:

1. si considera la prima **Affected Version** riportata nel sistema di tracciamento, ad esempio in Jira;
2. se questa informazione non è disponibile, si utilizza l'algoritmo **SZZ**.

Nota del Prof

Dal punto di vista pratico, si prendono i ticket chiusi e classificati come difetti, si individuano i commit che li correggono e si etichettano come difettose le classi toccate da quei commit, cercando poi di risalire all'indietro fino al momento di introduzione del bug.

9.4 SZZ: funzionamento e limiti

9.4.1 Funzionamento

SZZ è un processo algoritmico che cerca di identificare quando un difetto è stato materialmente introdotto in un progetto software. Il suo funzionamento si basa sui meccanismi di annotazione del sistema di versionamento, come `git blame`.

L'idea è la seguente: a partire dalle righe modificate in un **bug-fix commit**, SZZ cerca di determinare quando quelle stesse righe erano state modificate l'ultima volta prima del fix. In questo modo tenta di risalire al **bug-introducing change**.

9.4.2 Limiti

Un primo limite riguarda le **code additions**. Se il fix di un bug consiste nell'aggiungere codice mancante, SZZ può fallire perché non esiste una precedente modifica della stessa riga da tracciare.

Solo nelle slide (non spiegato a voce)

SZZ riesce a trovare solo il **26–29%** dei **bug-introducing changes**, perché si basa sull'assunzione che il bug sia stato introdotto nelle stesse linee di codice modificate per correggerlo.

9.4.3 Spiegazione del diagramma mostrato nelle slide

Il diagramma riportato nelle slide mostra un caso in cui, nella versione V3, un programmatore corregge un ciclo cambiando una condizione da $i \leq 4$ a $i < 4$ per risolvere il **Bug #123**. Confrontando il fix con la versione precedente tramite una *diff*, SZZ percorre a ritroso la storia di quella riga e individua in **V2** il cambiamento che ha introdotto il difetto.

9.5 Obiettivi dello studio e Research Questions

Lo studio si propone di quantificare l'impatto del rumore introdotto dallo snoring nei dataset di defect prediction. Le domande di ricerca sono cinque:

1. **To what extent do defects sleep?**
2. **To what extent do classes snore?**
3. **To what extent does snoring impact the evaluation of classifiers accuracy?**
4. **To what extent does snoring impact defect prediction accuracy?**
5. **To what extent is no data better than snoring data?**

Le prime due RQ mirano a misurare il fenomeno, mentre le successive valutano il suo impatto sia sulla stima delle prestazioni dei classificatori sia sulla qualità effettiva della previsione.

9.6 RQ1: To what extent do defects sleep?

9.6.1 Rationale

L'obiettivo è comprendere quante release trascorrono tra l'introduzione di un difetto e la sua correzione. In questo modo si misura l'entità del fenomeno dei **difetti dormienti**, che restano invisibili a chi costruisce il dataset.

9.6.2 Design

Sono stati selezionati **20 grandi progetti Apache**, tracciati in **Jira** e versionati con **Git**, con almeno **10 release** e un'elevata qualità nel collegamento tra bug e codice.

9.6.3 Variabili e metriche

- **Variabile indipendente:** nome del bug in Jira.
- **Variabile dipendente:** numero di release per cui il bug ha dormito, misurato come distanza tra il momento di introduzione e quello di fix, sia in valore assoluto sia in percentuale rispetto al numero totale di release.

9.6.4 Risultati

I **boxplot** mostrano che la distribuzione del numero di release dormite è significativamente spostata rispetto allo zero, segnalando che il fenomeno non è marginale ma strutturale.

9.6.5 Discussione

In media, un bug dorme per **7 release**. Inoltre, nella maggior parte dei progetti, un difetto dorme per oltre il **19%** delle release esistenti.

Solo nelle slide (non spiegato a voce)

Esistono anche casi estremi, come il bug **LUCENE-3672**, che ha dormito per **84 release**, e **STRATOS-1653**, rimasto dormiente per **51 release su 53**.

9.7 RQ2: To what extent do classes snore?

9.7.1 Rationale

Un difetto che dorme non produce automaticamente una classe che russa. Se nella stessa classe esiste almeno un altro difetto già noto, la classe viene comunque etichettata come difettosa. Per questo motivo, l'obiettivo non è solo contare i bug dormienti, ma misurare quanto il fenomeno si traduca davvero in **snoring classes**.

9.7.2 Design

L'idea sperimentale consiste nell'osservare un sottoinsieme iniziale del dataset e misurare come cambia la stima della difettosità man mano che il tempo avanza. Nelle slide questo meccanismo è visualizzato tramite i **binocoli**, che rappresentano l'osservatore spostato in avanti nel tempo.

9.7.3 Variabili e metriche

- **Variabile indipendente:** **Progress**, cioè il rapporto tra release osservata e numero totale di release del progetto.
- **Variabile dipendente:** **Missing Rate**, definito come:

$$\frac{FN}{FN + TP}$$

cioè la quota di classi realmente difettose che non vengono riconosciute come tali. Questa quantità corrisponde a **1 - Recall**.

9.7.4 Risultati

Il grafico a linee mostra chiaramente che, all'aumentare del **Progress**, il **Missing Rate** tende a scendere fino ad avvicinarsi allo zero. Ciò significa che serve tempo affinché il dataset riesca a riconoscere correttamente i difetti già presenti.

9.7.5 Discussione

Per la maggior parte dei progetti, il **Missing Rate** resta superiore al **50%** a meno che non si elimini oltre il **20%** delle release più recenti.

Nota del Prof

Per ottenere un **Missing Rate nullo**, in media bisogna ignorare circa il **71%** finale delle release. Questo mostra che la misurazione della difettosità ha bisogno di molto tempo per stabilizzarsi.

9.8 RQ3: To what extent does snoring impact the evaluation of classifiers accuracy?

9.8.1 Rationale

Ricercatori e professionisti confrontano costantemente classificatori, tecniche di feature selection e strategie di tuning. Se però i dataset usati per la valutazione sono contaminati dallo snoring, allora anche le conclusioni sperimentali risultano **biasate**.

9.8.2 Design

Sono stati impiegati **15 classificatori** differenti. La validazione è stata effettuata con una strategia di **ordered holdout**, mantenendo l'ordine cronologico dei dati: **66%** per il training e **33%** per il testing.

9.8.3 Variabili e metriche

Le variabili dipendenti considerate sono:

- **TP, TN, FP, FN;**
- **Precision;**
- **Recall;**
- **F1;**
- **Cohen's Kappa;**
- **AUC;**
- **Matthews Correlation Coefficient.**

Inoltre viene calcolato il **Relative Bias**, cioè la distanza relativa tra l'accuratezza di un classificatore misurata su dati rumorosi e quella misurata su dati privi di rumore.

La **variabile indipendente** è la presenza o assenza dello **snoring** nei set di training e di testing.

9.8.4 Risultati

I boxplot mostrano un **Relative Bias** molto elevato per quasi tutte le metriche. I valori medi riportati sono particolarmente alti per **Recall**, **Kappa** e **Matthews**, mentre risultano sensibilmente più bassi per **AUC**.

9.8.5 Discussione

Lo snoring può portare a premiare il classificatore sbagliato.

Solo nelle slide (non spiegato a voce)

Quando la valutazione viene eseguita su dati affetti da snoring, il classificatore realmente migliore viene selezionato correttamente solo nel **45%** dei casi.

9.9 RQ4: To what extent does snoring impact defect prediction accuracy?

9.9.1 Rationale

Dopo aver mostrato che lo snoring altera la valutazione dei classificatori, occorre misurare quanto esso peggiori anche la loro **accuratezza predittiva reale**.

9.9.2 Design

L'accuratezza viene testata addestrando i modelli su dati rumorosi, indicati come **TrS**, e valutandoli su un test set privo di snoring, indicato come **TeNS**. In questo modo il deterioramento osservato dipende dalla qualità del training set e non da un rumore condiviso anche nel test.

9.9.3 Variabili e metriche

Le variabili e le metriche sono le stesse della RQ3.

9.9.4 Risultati

Il confronto tra i boxplot mostra un peggioramento sistematico delle prestazioni. La degradazione interessa tutti i classificatori e tutte le metriche osservate.

9.9.5 Discussione

Lo snoring degrada pesantemente le performance di tutti i **15 classificatori**. La metrica meno colpita risulta essere la **Precision**.

IMPORTANTE PER L'ESAME

La **Recall** crolla mentre la **Precision** si riduce molto meno perché lo snoring introduce soprattutto **Falsi Negativi**. Se il modello viene addestrato con dati che fanno apparire molte classi difettose come non difettose, tenderà a essere molto conservativo e a prevedere quasi sempre la classe pulita. In questo modo mancherà moltissimi veri difetti, facendo precipitare la **Recall**; tuttavia, le poche volte in cui segnalerà un difetto, tenderà a farlo su casi relativamente più solidi, mantenendo la **Precision** più stabile.

9.10 RQ5: To what extent is no data better than snoring data?

9.10.1 Rationale

Poiché non è possibile eliminare con precisione solo i singoli datapoint rumorosi, una possibile strategia consiste nel rimuovere intere release recenti, che sono le più esposte al rischio di contenere difetti ancora dormienti. La domanda è quindi se sia preferibile avere **meno dati ma più puliti**.

9.10.2 Design

L'esperimento confronta diversi scenari in cui vengono rimosse le ultime **1, 2, 3 o 4 release** dal training set. Le classi già esplicitamente etichettate come difettose vengono mantenute, perché non possono rappresentare falsi negativi.

9.10.3 Variabili e metriche

- **Variabile indipendente:** numero di release eliminate.
- **Variabile dipendente:** guadagno relativo sulle metriche di accuratezza.

9.10.4 Risultati

Rimuovere l'ultima release produce in media un miglioramento significativo, in particolare su **Recall** e **F1**. Al contrario, rimuovere tre o quattro release porta a un peggioramento delle prestazioni.

9.10.5 Discussione

Ignorare una sola release è generalmente meglio che usare l'intero dataset. Tuttavia, eliminare troppe release impoverisce eccessivamente il training set.

Nota del Prof

Qui emerge il conflitto tra due principi ingegneristici. Da un lato c'è il **Garbage In, Garbage Out**: se si mantengono dati freschi ma rumorosi, il modello viene contaminato. Dall'altro c'è il principio per cui **la dimensione del dataset conta**: se si eliminano troppe release, il modello perde informazione utile e peggiora. Il compromesso migliore, cioè lo *sweet spot*, consiste nello scartare soltanto le ultime **1 o 2 release**.

9.11 Conclusioni

Dallo studio emerge che il fenomeno dei **sleeping defects** e delle **snoring classes** ha un impatto molto forte sulla costruzione dei dataset di defect prediction.

- la maggior parte dei difetti dorme per oltre il **19%** del ciclo di vita del progetto;
- il **Missing Rate** resta superiore al **50%** finché non si rinuncia ad analizzare almeno il **20%** finale delle release;
- l'errore relativo nella valutazione dei classificatori si aggira intorno al **100%** per quasi tutte le metriche, con l'eccezione dell'**AUC**;
- lo snoring penalizza in particolare **Recall**, **F1**, **Matthews** e **Cohen's Kappa**;
- rimuovere l'ultima release può migliorare sensibilmente la qualità della prediction.

9.12 Future works

Le possibili estensioni del lavoro includono:

- lo sviluppo di tecniche di rimozione del rumore più **fine-grained**;
- la combinazione di più tecniche di mitigazione del rumore;
- l'uso di distribuzioni probabilistiche come input dei modelli predittivi, invece di etichette binarie rigide;
- la replica dello studio nel contesto della classificazione **Just-In-Time (JIT)**.

Capitolo 10

Effort-Aware Accuracy Metrics e valutazione dei classificatori

10.1 Contesto generale

Nel campo dell'**ingegneria del software**, i modelli di **defect prediction** sono classificatori di Machine Learning con lo scopo di identificare gli artefatti software, come classi o metodi, che hanno un'alta probabilità di manifestare un difetto.

10.1.1 Defect prediction e testing activity context

Il fine ultimo della **defect prediction** è ridurre il costo delle attività di testing, analisi o *code review*. Prevedendo dove si annidano i bug, il team di sviluppo può dare priorità agli sforzi, concentrandoli su porzioni specifiche di codice, invece di ispezionare l'intero sistema alla cieca.

10.1.2 Perché l'accuratezza classica non basta

Storicamente, i modelli predittivi venivano valutati tramite metriche classiche derivate dal mondo dell'**Information Retrieval**, come **Precision** o **Recall**.

Nota del Prof

Il professore sottolinea che, sebbene le metriche classiche siano utili a livello teorico, risultano inadeguate e poco pratiche nel contesto del testing. Sapere che un classificatore ha una Precision dello 0.90 o una Recall dello 0.89 non fornisce al manager alcuna indicazione pratica sull'impatto reale in termini di *effort* necessario per ispezionare il software. Per questo motivo, le metriche classiche risultano troppo astratte.

10.1.3 Effort-aware accuracy metrics

Per colmare questo divario, sono nate le **Effort-Aware Accuracy Metrics (EAM)**. Si tratta di metriche di accuratezza che tengono esplicitamente in considerazione l'**effort**, cioè il costo o il tempo richiesto per ispezionare il codice suggerito dal classificatore, garantendo così una stima più realistica dei benefici apportati.

10.2 Concetti fondamentali

10.2.1 Definizione di EAM

Le **Effort-Aware Metrics** misurano i benefici forniti da un classificatore in base alla sua capacità di ordinare correttamente le entità candidate a essere difettose, bilanciando i bug trovati con lo sforzo speso per cercarli.

10.2.2 Ranking delle entità difettose

Poiché è impossibile analizzare tutto il software, file e classi vengono inseriti in un **ranking**, cioè una lista ordinata. Il tester ispezionerà il codice partendo dalla cima della lista e scendendo verso il basso finché non esaurisce le risorse a disposizione.

10.2.3 Relazione tra effort e LOC da ispezionare

Nota del Prof

Nell'approccio EAM si assume, come approssimazione, che l'effort necessario per testare un'entità sia linearmente proporzionale alla sua dimensione fisica, cioè al numero di **Linee di Codice (LOC)**. Ispezionare una classe enorme costerà quindi molto più effort rispetto a una classe molto piccola.

10.2.4 Differenza tra ranking per probabilità e ranking effort-aware

L'approccio classico ordina semplicemente le classi in base alla **probabilità** di contenere un bug. Un approccio realmente **effort-aware**, invece, ordina le classi tenendo conto anche di quanto *costa* ispezionarle in termini di LOC.

10.3 PofBx e NPofBx

10.3.1 Definizione di PofBx

La **PofBx** è la EAM non normalizzata più nota. Definisce la percentuale di bug che un tester può identificare ispezionando il top $x\%$ delle linee di codice del sistema, seguendo un ranking basato esclusivamente sulla **probabilità predetta** di difetto.

10.3.2 Definizione di NPofBx

La **NPofBx** è la metrica proposta dagli autori del paper. Rappresenta la versione **normalizzata** della PofBx. Mantiene la stessa logica generale, ma utilizza un ranking differente.

10.3.3 Significato della normalizzazione

Invece di ordinare semplicemente per probabilità predetta, la NPofBx usa un ranking basato sulla **predicted defect density**, cioè una densità predetta dei difetti.

10.3.4 Ranking per predicted probability / size

La densità viene calcolata dividendo la probabilità predetta di un'entità per la sua dimensione:

$$\frac{\text{predicted probability}}{\text{size}}$$

10.3.5 Differenza pratica tra i due approcci

Nota del Prof

Senza normalizzazione, il modello potrebbe suggerire di testare per prima una classe enorme con probabilità del 90% di contenere un bug, esaurendo subito il budget di LOC analizzabili. Con la normalizzazione, il modello capisce che è più conveniente testare per prime diverse classi piccole ma promettenti, trovando molti più bug a parità di LOC lette.

10.4 Esempio introduttivo

Per chiarire l'effetto della normalizzazione, le slide e l'audio presentano un esempio pratico.

10.4.1 Dataset di esempio

Si considera un sistema di **1790 LOC**, diviso in **7 entità**, di cui **3** effettivamente difettose.

10.4.2 Significato di PofB20

Si assume di poter utilizzare il 20% del budget di ispezione. Il 20% di 1790 LOC corrisponde a **358 LOC**. Ordinando le 7 entità per semplice **probabilità predetta**, le prime due entità consumano tutto il budget disponibile. Analizzandole, si trova solo **1** delle **3** entità difettose. Di conseguenza:

$$PofB20 = 33\%$$

10.4.3 Significato di NPofB20

Applicando la normalizzazione, cioè ordinando le entità per Probabilità/Size, le classi piccole e promettenti risalgono la graduatoria. Con lo stesso tetto di 358 LOC, si riescono ad analizzare **3** entità anziché 2, trovando **2** delle **3** entità difettose. Di conseguenza:

$$NPofB20 = 66\%$$

10.4.4 Perché la normalizzazione migliora il numero di difetti trovati

Semplicemente cambiando il criterio di ordinamento del ranking, senza alterare i dati originali né il classificatore, il numero di bug trovati **raddoppia**.

10.5 Aim del lavoro e Research Questions

10.5.1 Problemi nell'uso delle EAM

Il lavoro nasce dall'osservazione che esiste una grave carenza, e talvolta un uso scorretto, delle EAM in letteratura, il che ne mina la validità e la generalizzazione.

10.5.2 Proposta di normalizzazione

L'obiettivo del lavoro è duplice:

1. evidenziare le problematiche storiche nell'uso delle EAM;
2. proporre e valutare la validità della metrica normalizzata **NPofB**, basata sulla **predicted defect density**.

10.5.3 Research Questions

Le quattro domande di ricerca sono:

- **RQ1**: quali EAM vengono usate nei paper di Ingegneria del Software?
- **RQ2**: la normalizzazione migliora effettivamente i valori di PofBs?
- **RQ3**: il ranking dei classificatori cambia se si normalizza la PofB?
- **RQ4**: il ranking dei classificatori cambia al variare della x nelle NPofBx normalizzate?

10.6 RQ1

Per rispondere alla prima domanda, gli autori hanno effettuato uno **Systematic Mapping Study**.

10.6.1 Numero di studi, progetti, EAM e classifier

Sono stati analizzati **152 primary studies** provenienti dai principali journal del settore. Lo studio finale è stato validato su **14 progetti** — 12 open-source e 2 industriali — confrontando **10** diverse soglie di EAM e usando **10 classificatori**.

10.6.2 Risultati principali

I risultati mostrano che:

- la grande maggioranza della letteratura ignora le EAM: **132 studi su 152** usano **0 EAM**, valutando i modelli solo tramite Precision o Recall;
- i pochi studi che usano EAM si limitano quasi sempre a **una sola metrica**, tipicamente **PofB20**, riducendo la generalizzabilità dei risultati;
- sette studi su venti presentavano un **mismatch nei nomi delle metriche**, sintomo di scarsa comprensione o assenza di standardizzazione.

10.7 RQ2

10.7.1 Intuizione di base

La RQ2 indaga se la **NPofB** ottenga performance matematicamente migliori rispetto alla **PofB**.

10.7.2 Variabili indipendente e dipendente

Nota del Prof

Il professore richiama una nozione cruciale di ingegneria degli esperimenti. In questo studio, la **variabile indipendente** è la presenza o assenza della normalizzazione, mentre la **variabile dipendente** è lo score della PofB calcolato con o senza normalizzazione.

10.7.3 PofB10–PofB90 e average

La valutazione è stata eseguita su un intero spettro di soglie, dal **10%** al **90%** a passi di 10, escludendo 0 e 100 perché banali, più una metrica media, **AveragePofB**. In totale vengono considerate **10 PofBs**.

10.7.4 Preprocessing

Prima dell'esperimento, i dati sono stati preprocessati tramite:

1. **Log10 normalization**;
2. **Feature subset selection**;
3. **SMOTE**.

Nota del Prof

La trasformazione in $\log_{10}$ è fondamentale. Se si usano feature disomogenee, per esempio secondi, numero di bug o LOC, passare al logaritmo base 10 rende le distribuzioni più omogenee e aumenta l'accuratezza di base.

10.7.5 Walk-forward

Nota del Prof

Il **walk-forward** è vitale nell'ingegneria del software. Si dividono i dati in release sequenziali: per predire la Release 4 si usa come training set l'unione delle release 1, 2 e 3. Questo impedisce al modello di leggere il futuro e preserva il realismo temporale.

10.7.6 Classifier usati

Solo nelle slide (non spiegato a voce)

Sono stati usati 10 algoritmi ML ben noti: **Decision Table**, **IBk (k-NN)**, **J48**, **KStar**, **Naive Bayes**, **SMO**, **Random Forest**, **Logistic Regression**, **BayesNet** e **Bagging**.

10.7.7 Risultati

La normalizzazione incrementa i valori della PofB in modo statisticamente significativo e di svariati ordini di grandezza.

10.7.8 Relative gain

Il guadagno relativo viene calcolato come:

$$\frac{NPofB - PofB}{PofB}$$

Il **relative gain** cresce fortemente per soglie più basse: per esempio, il guadagno della **PofB10** è circa doppio rispetto a quello della **PofB20**.

10.7.9 Cliff's delta e p-value

IMPORTANTE PER L'ESAME

Il professore insiste molto sulla differenza tra **p-value** ed **effect size**. Il **p-value** indica quanto sia probabile commettere errore nel rigettare l'ipotesi nulla, ma *non* misura la grandezza dell'effetto. Il **Cliff's delta**, invece, misura proprio quanto l'effetto sia grande e importante.

Nota del Prof

Nel caso della normalizzazione, i valori di **Cliff's delta** risultano per lo più classificati come **Large**, cioè maggiori o uguali a 0.43, mostrando che l'effetto non è solo statisticamente significativo, ma anche sostanzialmente rilevante.

10.8 RQ3

10.8.1 Spearman rank correlation

La RQ3 si chiede se, normalizzando la PofB, il classificatore che prima risultava il migliore resti tale oppure se la graduatoria venga stravolta. Per confrontare i ranking si usa il coefficiente di correlazione di **Spearman**.

10.8.2 Confronto ranking con e senza normalizzazione

Il confronto viene fatto tra le classifiche dei classificatori ottenute con la stessa PofB, ma calcolata **con** e **senza** normalizzazione.

10.8.3 Best classifier

L'analisi si concentra anche su un aspetto pratico fondamentale: verificare se il **miglior classificatore** resti lo stesso prima e dopo la normalizzazione.

10.8.4 Risultati e interpretazione pratica

I risultati mostrano una correlazione solo **modesta** nella maggior parte dei casi. In pratica, identificare il miglior classificatore usando la vecchia PofB equivale spesso a scegliere il classificatore sbagliato rispetto alla più realistica NPofB.

Questo implica che molti studi passati basati sulla PofB non normalizzata risultano fuorvianti, perché potrebbero aver promosso come *miglior modello* un algoritmo che, in realtà, non fornisce il massimo beneficio effort-aware al tester.

10.9 RQ4

10.9.1 Confronto tra diverse NPofBx

Una volta stabilito che la normalizzazione è opportuna, la RQ4 si chiede se basti usare una sola metrica, per esempio **NPofB20**, oppure se il comportamento dei classificatori cambi al variare della soglia x .

10.9.2 Ruolo di x da 10 a 90

L'analisi considera soglie che vanno dal **10%** al **90%**, oltre alla metrica media **Average NPofB**.

10.9.3 Average NPofB e confronto dei ranking

Le diverse classifiche vengono confrontate a coppie tramite **Spearman**, valutando anche il comportamento dell'**Average NPofB**.

10.9.4 Risultati e best classifier

I risultati dimostrano che i ranking dei classificatori sono molto lontani dall'essere identici al variare della soglia x . In circa l'**88%** dei casi, il classificatore migliore cambia se si cambia la quantità di codice da ispezionare.

In altre parole, il modello ottimale per ispezionare il **10%** del codice potrebbe non essere più il migliore se l'obiettivo è ispezionarne il **50%**.

10.10 Figure, tabelle e grafici

Solo nelle slide (non spiegato a voce)

Le tabelle e i grafici descritti qui sono presentati nelle slide come supporto visivo dei risultati, anche quando non vengono spiegati in dettaglio a voce.

10.10.1 Esempio PofB20 e NPofB20

Le tabelle iniziali del paper illustrano visivamente l'esempio dei 7 file, mostrando il passaggio da **33%** a **66%** di bug trovati grazie alla normalizzazione.

10.10.2 Tabella delle EAM usate negli studi precedenti

La **Tabella 4.4** mostra che, su **152** studi primari, **132** usano **0 EAM**. Nessun paper in letteratura ha usato **5 o più EAM**.

10.10.3 Tabella dei relative gains

La **Tabella 4.6** mostra il forte guadagno dovuto alla normalizzazione:

- +414% per **PofB10**;
- guadagni progressivamente decrescenti fino a +11% per **PofB90**;
- tutti i confronti hanno **p-value < 0.0001**.

10.10.4 Tabella degli equivalent best classifiers

La **Tabella 4.8** e il boxplot associato mostrano la proporzione di casi in cui il miglior modello resta lo stesso. Per esempio, si passa dal **15%** per **NPofB10** al **29%** per **NPofB90**. Questo dato è importante perché smonta l'idea che testare una sola EAM sia statisticamente e industrialmente corretto.

10.11 Main results e conclusioni

10.11.1 Superiorità delle NPofBs rispetto alle PofBs

Dal punto di vista statistico e dell'ordine di grandezza, la **NPofB** risulta nettamente superiore alla **PofB**. Fornisce valutazioni più realistiche perché allinea le stime matematiche al comportamento effettivo di uno sviluppatore, che nella pratica considera sempre anche il peso delle dimensioni del file.

10.11.2 Cambiamento del best classifier dopo la normalizzazione

L'impatto più forte della normalizzazione è che essa **ribalta il ranking dei classificatori**. Modelli ritenuti ottimi nella letteratura precedente perdono il primato quando si passa alla valutazione normalizzata.

10.11.3 Implicazioni pratiche

Chiunque valuti un classificatore in ingegneria del software dovrebbe:

- calcolare metriche **Effort-Aware normalizzate**, cioè **NPofB**;
- testare il classificatore su **molteplici soglie x** , e non su una sola;
- evitare conclusioni basate esclusivamente su una singola metrica o su una sola soglia.

10.12 ACUME tool

10.12.1 Scopo

L'assenza di metriche EAM in letteratura è spesso dovuta alla mancanza di librerie standard per calcolarle. Gli autori propongono quindi **ACUME** (**ACcUracy MEtrics**), un tool open-source in Python per automatizzare il processo.

10.12.2 Input e output

Il tool agisce alla fine della pipeline. Prende in input un file **CSV** predetto da strumenti di Machine Learning, come Weka o scikit-learn, con quattro colonne essenziali:

- **ID**
- **Size**
- **Predicted Probability**
- **Actual Defectiveness**

In output produce un singolo file CSV contenente:

- le metriche classiche di accuratezza;
- tutte le EAM discusse nel lavoro.

10.12.3 Utilità per i ricercatori

ACUME evita sprechi di tempo, riduce il rischio di mismatch nei nomi delle metriche, aumenta la generalizzazione dei risultati e migliora la riproducibilità degli esperimenti.

In sintesi

Le metriche classiche, come Precision e Recall, risultano troppo astratte nel mondo reale del testing software. Questo studio mostra che pesare il ranking dividendo la probabilità per la grandezza del file, cioè usando **NPofB**, aumenta drasticamente la capacità di ritrovare i bug veri. Il risultato è un cambiamento profondo nelle modalità con cui l'accademia e i professionisti dovrebbero valutare i classificatori: mai più con una sola soglia e mai più senza normalizzazione.

Capitolo 11

Snoring: rumore nei dataset di defect prediction

11.1 Introduzione e contesto della defect prediction

La **defect prediction** è una tecnica che mira a identificare gli artefatti software, come le classi, che hanno un'alta probabilità di presentare un difetto.

- **Obiettivo:** ridurre i costi e il tempo dedicati alle fasi di testing e code review, permettendo agli sviluppatori di concentrare gli sforzi su specifiche parti critiche del codice.
- **Contesto:** lo studio si posiziona nell'ambito della **within-project defect prediction**, ovvero la classificazione delle classi difettose all'interno di una specifica release di un progetto.
- **Problema dei dati:** l'affidabilità di un modello predittivo dipende interamente dalla qualità del dataset su cui viene addestrato.

Nota del Prof

Questo richiama il principio del **Garbage In, Garbage Out**. Se i dati in ingresso sono inaccurati, i risultati del classificatore saranno inaffidabili.

Solo nelle slide (non spiegato a voce)

Lavori precedenti hanno già identificato altre fonti di rumore nei dataset, come la **misclassification** e l'origine dei difetti, proponendo soluzioni mirate.

11.2 Sleeping defects e snoring classes

- **Sleeping defects (difetti dormienti):** è possibile che un difetto venga scoperto o corretto solo diverse release dopo la sua introduzione, cioè come **Post-Release Defects**. Durante questo intervallo, il difetto “dorme” nel codice.
- **Snoring classes (classi che russano):** è il fenomeno per cui alcune classi sono affette *esclusivamente* da difetti non ancora corretti. Poiché la presenza di un difetto

non può essere nota prima del suo fix, queste classi appaiono erroneamente come pulite. Contenendo difetti dormienti, producono quindi **rumore** nei dataset di defect prediction.

Nota del Prof

La metafora è semplice: il difetto dorme, ma è la classe che “rusa”, creando disturbo. Se una classe ha cinque bug dormienti ma anche un solo bug già identificato, la classe viene comunque etichettata come difettosa e non russa. La classe russa solo quando **tutti** i suoi bug sono dormienti, sfuggendo all’etichettatura.

IMPORTANTE PER L’ESAME

Lo **snoring** introduce nei dataset **solo Falsi Negativi (FN)** e mai Falsi Positivi. Una classe etichettata come difettosa lo è per certo, perché il fix esiste; una classe etichettata come non difettosa potrebbe invece nascondere un difetto dormiente. Per questo motivo, l’ultima release di un software sembrerà sempre perfetta: i bug ci sono, ma non sono ancora stati trovati.

11.3 Come vengono assegnati i difetti alle classi

Per capire da quando una classe è difettosa, si cerca di risalire al momento di introduzione del bug:

1. si considera la prima **Affected Version** riportata nel sistema di tracciamento, ad esempio Jira;
2. se questa informazione manca, si utilizza l’algoritmo **SZZ**.

Nota del Prof

Dal punto di vista pratico, si prendono i ticket chiusi e classificati come difetti, si individuano i commit che li risolvono e si etichettano come difettose le classi toccate da quel commit, andando a ritroso nel tempo fino alla data di iniezione.

11.4 SZZ: funzionamento e limiti

11.4.1 Funzionamento

SZZ è un processo algoritmico per identificare quando un difetto è stato materialmente introdotto nel progetto. Sfrutta il meccanismo di annotazione del sistema di versionamento, come `git blame`, per determinare, partendo dalle righe di codice modificate durante il fix di un bug, quando quelle stesse righe erano state alterate in precedenza.

11.4.2 Limiti

- **Code Additions:** se il fix consiste nell’aggiungere codice mancante, **SZZ** fallisce perché si basa sull’assunzione che il bug sia stato introdotto nelle stesse linee di codice toccate per correggerlo.

Solo nelle slide (non spiegato a voce)

SZZ riesce a trovare solo il **26–29%** dei **bug-introducing changes**, proprio a causa di questa assunzione restrittiva.

11.4.3 Spiegazione del diagramma SZZ nelle slide

Il diagramma mostra un esempio pratico. In **V3**, che rappresenta il fix, il programmatore corregge un ciclo da $i \leq 4$ a $i < 4$ per risolvere il **Bug #123**. Tramite una *diff* con la versione precedente, SZZ traccia all'indietro l'origine di quella riga di codice, identificando **V2** come la modifica che ha introdotto l'errore.

11.5 Obiettivi dello studio e Research Questions

Lo studio si propone di quantificare l'impatto di questo rumore e si articola in cinque **Research Questions (RQ)**:

1. **To what extent do defects sleep?**
Quanto a lungo dormono i difetti?
2. **To what extent do classes snore?**
In che misura le classi russano?
3. **To what extent does snoring impact the evaluation of classifiers accuracy?**
Come impatta la valutazione dei classificatori?
4. **To what extent does snoring impact defect prediction accuracy?**
Come impatta l'accuratezza predittiva?
5. **To what extent is no data better than snoring data?**
Fino a che punto “nessun dato” è meglio di “dati che russano”?

11.6 RQ1: To what extent do defects sleep?

11.6.1 Rationale

L'obiettivo è comprendere quante release trascorrono tra l'iniezione di un difetto e la sua correzione, misurando i difetti dormienti ignoti a chi crea il dataset.

11.6.2 Design

Sono stati selezionati **20 grandi progetti Apache**, tracciati su **Jira** e versionati su **Git**, con almeno **10 release** ed elevato tracciamento tra bug e codice.

11.6.3 Variabili e metriche

- **Variabile indipendente:** nome del bug in Jira.
- **Variabile dipendente:** numero di release dormite, misurate tra iniezione e fix, sia in valore assoluto sia in percentuale rispetto alle release totali.

11.6.4 Risultati

I diagrammi a scatola (**boxplot**) mostrano la distribuzione temporale del fenomeno. Le mediane evidenziano un forte scostamento rispetto allo zero.

11.6.5 Discussione

Il bug medio dorme per ben **7 release**. Nella maggior parte dei progetti, un difetto dorme per più del **19%** delle release esistenti del progetto.

Solo nelle slide (non spiegato a voce)

Esistono casi estremi come il bug **LUCENE-3672**, dormiente per **84 release**, causato da un cambio di librerie esterne.

11.7 RQ2: To what extent do classes snore?

11.7.1 Rationale

Un difetto che dorme non genera automaticamente una classe che russa se un altro difetto nella stessa classe viene scoperto. L'obiettivo è quindi misurare quanto effettivamente le classi siano affette dallo **snoring**.

11.7.2 Design

Si misura come varia la misurazione analizzando un sottoinsieme “sterile”, cioè il primo **5%** del dataset, osservandolo dal futuro. Lo schema nelle slide utilizza l'icona dei **binocoli** per mostrare l'osservatore che si sposta temporalmente in avanti.

11.7.3 Variabili e metriche

- **Variabile indipendente: Progress**, cioè il progresso temporale, pari alla release misurata divisa per il numero totale di release.
- **Variabile dipendente: Missing Rate**, che equivale ai Falsi Negativi, ossia la percentuale di classi difettose che non vengono identificate come tali:

$$\frac{FN}{FN + TP}$$

Questa quantità equivale a **1 - Recall**.

11.7.4 Risultati

Il grafico a linee mostra chiaramente che, all'aumentare del **Progress**, il **Missing Rate** precipita verso lo zero.

11.7.5 Discussione

Per la maggior parte dei progetti, il **Missing Rate** è superiore al **50%** a meno che non si rimuova più del **20%** delle release recenti.

Nota del Prof

Per non avere alcun Missing Rate, cioè per arrivare a **0%**, in media bisogna ignorare l'ultimo **71%** delle release. Questa è la prova che la misurazione richiede molto tempo per stabilizzarsi.

11.8 RQ3: To what extent does snoring impact the evaluation of classifiers accuracy?

11.8.1 Rationale

Ricercatori e praticanti valutano costantemente tecnologie e classificatori. Se lo **snoring** impatta questi test, allora anche le valutazioni riportate in letteratura risultano compromesse da un forte **bias**.

11.8.2 Design

Sono stati impiegati **15 algoritmi di classificazione** differenti. Si è usata la tecnica dell'**holdout** conservando l'ordine cronologico dei dati: **66%** dei dati storici per il training e **33%** per il testing.

11.8.3 Variabili e metriche

- **Variabili dipendenti:** TP, FN, TN, FP.
- **Metriche complesse:** Precision, Recall, F1, Cohen's Kappa, AUC, Matthews.
- **Misura aggiuntiva:** **Relative Bias**, cioè la distanza relativa tra l'accuratezza con rumore e quella senza rumore.
- **Variabile indipendente:** presenza o assenza di snoring nel training e nel testing.

11.8.4 Risultati

I boxplot mostrano un **Relative Bias** altissimo, vicino al **100%** per quasi tutte le metriche. Tra i valori riportati spiccano **Recall = 1.11** e **Kappa = 1.21**, mentre l'**AUC** è molto meno colpita, con valore **0.11**.

11.8.5 Discussione

A causa dello snoring, si rischia di premiare algoritmi sbagliati.

Solo nelle slide (non spiegato a voce)

Valutando su dati affetti da snoring, il classificatore che è realmente il migliore in un progetto viene selezionato correttamente solo nel **45%** dei casi.

11.9 RQ4: To what extent does snoring impact defect prediction accuracy?

11.9.1 Rationale

Stabilito che lo **snoring** è una forma di rumore, si vuole quantificare matematicamente quanto peggiorino i modelli predittivi usati per trovare i bug.

11.9.2 Design

Si testa l'accuratezza di un modello addestrato su dati sporchi, indicati come **TrS** (*Training con Snoring*), contro un test set asettico e privo di rumore, indicato come **TeNS** (*Test No Snoring*).

11.9.3 Variabili e metriche

Le variabili e le metriche considerate sono identiche a quelle della RQ3.

11.9.4 Risultati

Confrontando le distribuzioni nei grafici a scatola, l'accuratezza statistica crolla in modo evidente.

11.9.5 Discussione

Lo **snoring** degrada in maniera drammatica le performance per tutti i **15 classificatori** e su tutte le metriche. La metrica meno impattata in assoluto è la **Precision**.

IMPORTANTE PER L'ESAME

La **Recall** viene demolita mentre la **Precision** subisce danni più lievi perché lo **snoring** nasconde classi difettose etichettandole come pulite, creando moltissimi **Falsi Negativi**. Se addestriamo un modello fornendogli quasi esclusivamente esempi di classi pulite, il modello diventa conservativo e tende a predire quasi sempre “non difettoso”. Predicendo pochissimi bug, mancherà quasi tutti i bersagli veri, facendo crollare la **Recall**; tuttavia, le poche volte in cui lancerà un allarme, tenderà a farlo su basi più solide, mantenendo relativamente stabile la **Precision**. Quando il dataset è pulito, invece, l'aumento dei positivi è reale e la Recall può risalire.

11.10 RQ5: To what extent is no data better than snoring data?

11.10.1 Rationale

Poiché non possiamo rimuovere chirurgicamente i singoli punti rumorosi, perché i bug sono invisibili finché non vengono scoperti, l'unica difesa consiste nell'eliminare intere release recenti, che sappiamo essere ricche di difetti non ancora emersi. La domanda è quindi se rimuovere dati migliori davvero le predizioni.

11.10.2 Design

Si testano i classificatori rimuovendo a scalare le ultime **1, 2, 3 o 4 release (DropCount)** dal set di addestramento. Le classi etichettate esplicitamente come difettose vengono invece preservate, poiché un difetto palese non può essere un falso negativo.

11.10.3 Variabili e metriche

- **Variabile indipendente:** numero di release eliminate.
- **Variabile dipendente:** guadagno relativo sulle metriche di accuratezza.

11.10.4 Risultati

Rimuovere l'ultima release, cioè **+1 DropCount**, garantisce in media un guadagno relativo del **+41%** in **Recall** e del **+33%** in **F1**. Rimuovere **3 o 4** release restituisce invece valori negativi.

11.10.5 Discussione

Ignorare una singola release è sempre meglio che usare tutti i dati. Ignorare **3 o 4** release causa l'effetto inverso e peggiora i modelli.

Nota del Prof

Qui emerge un conflitto ingegneristico tra il principio **Garbage In, Garbage Out**, secondo cui dati freschi ma rumorosi inquinano il modello, e il principio **Size Does Matter**, secondo cui rimuovere troppe release priva il modello di dati recenti utili per l'addestramento. Lo **sweet spot**, cioè il compromesso ideale, è scartare le ultime **1 o 2 release**.

11.11 Conclusioni

- La maggior parte dei difetti dorme per più del **19%** del ciclo di vita del progetto.
- Il **Missing Rate** resta sopra il **50%** finché non si accetta di ignorare l'ultimo **20%** delle release storiche.
- L'errore relativo nella valutazione dei classificatori si aggira intorno al **100%** per quasi tutte le metriche, ad eccezione dell'**AUC**.
- Il rumore dello snoring deprime pesantemente **Recall**, **F1**, **Matthews** e **Cohen's Kappa**.
- Applicare il **Drop** dell'ultima release migliora le predizioni di circa il **30%**.

11.12 Future works

- Sviluppo di metodologie di rimozione del rumore più **fine-grained**, cioè più granulari.
- Combinazione di diverse tecniche di mitigazione del rumore.
- Utilizzo di distribuzioni probabilistiche come input per i modelli predittivi al posto di etichette binarie nette, cioè difettoso/non difettoso.
- Replica dello studio nel contesto della classificazione **Just-In-Time (JIT)**.

INIZIO DEANGELIS

Capitolo 12

Panoramica del modulo di Software Testing

12.1 Introduzione al modulo

Questa è la lezione introduttiva al modulo di **Software Testing** di **Ingegneria del Software 2**. Il modulo ha l'obiettivo di mostrare come migliorare la qualità del software in produzione attraverso processi, metodologie e tecnologie di testing.

Il corso è tenuto da **Guglielmo De Angelis**, ricercatore presso il gruppo **SaKS** del **CNR (IASI)** di Roma. Le sue aree di ricerca includono:

- modellazione dei sistemi software;
- modellazione di proprietà extra-funzionali;
- generazione automatica a partire da modelli;
- architetture orientate ai servizi;
- software testing.

12.2 Informazioni organizzative

Le lezioni si tengono il **mercoledì**, dalle **11:30 alle 13:30**, in **Aula B14**, fino al **3 giugno 2026**.

Nota del Prof

Di comune accordo con l'aula, si faranno due ore di lezione consecutive, iniziando in orario e finendo puntuali, senza pause intermedie. Non sono previsti ponti o riduzioni del calendario.

12.2.1 Canali Teams

Sono previsti due canali principali:

- il **canale Generale**, usato per informazioni di carattere generale e per la condivisione delle slide;

- il **canale Software Testing**, dedicato alle domande specifiche sugli argomenti del modulo e sul progetto d'esame.

12.2.2 Ricevimento e comunicazione

Nota del Prof

È preferibile porre i propri dubbi direttamente in aula, così che tutti possano ascoltare le risposte. In alternativa, è fortemente consigliato usare la mailing list associata al canale Teams, in modo da discutere i problemi collettivamente anche nella lezione successiva.

IMPORTANTE PER L'ESAME

Il docente non legge in modo sincrono i messaggi diretti su Teams. Le richieste vanno fatte sempre via mail, preferibilmente mettendo in copia il canale Teams del modulo. Le mail private al docente devono essere usate solo per questioni strettamente personali.

Quando il corso non è attivo, il ricevimento avviene solo su appuntamento via email.

12.3 Sillabo del modulo

Il sillabo segue una progressione che parte dalle basi e arriva fino alla generazione automatica dei test:

- **Intro and General Concepts;**
- **Test Automation e Continuous Testing;**
- **Unit and Integration Testing;**
- **Approaches to Test Generation;**
- **Adequacy of the Tests: Control Flow Coverage;**
- **Adequacy of the Tests: Mutation Testing;**
- **Coverage-based Approaches to Test Generation.**

12.4 Materiale didattico

Solo nelle slide (non spiegato a voce)

Tra i testi di riferimento compaiono, tra gli altri, **A.P. Mathur** con *Foundations of software testing*, **B. Garcia** e **P. Tahchiev et al.** per la parte su **JUnit**, **W.E. Lewis** per il testing e il miglioramento continuo della qualità, e **J. Humble** con **D. Farley** per la *Continuous Delivery*.

Nota del Prof

Le slide sono sintesi drastiche dei concetti. Studiare solo sulle slide non basta: è necessario approfondire sui libri. Il testo di Mathur è molto utile per la parte teorica, mentre i libri su JUnit sono più vicini a una manualistica tecnica da consultare per la pratica.

12.5 Modalità di esame

Il lavoro richiesto è **strettamente individuale** e consiste nell'applicare le tecniche del corso a **due classi differenti** appartenenti a un **progetto open-source** della **Apache Software Foundation**.

I sorgenti del progetto devono essere disponibili su **GitHub** e lo studente deve configurare l'ambiente di lavoro del progetto.

Nota del Prof

Si consiglia fortemente di partire da un *fork* personale del progetto su GitHub. In questo modo, se durante il lavoro si individua un bug reale, è anche possibile aprire una *pull request* verso il progetto originale.

IMPORTANTE PER L'ESAME

È richiesta la configurazione di un framework di **Continuous Integration**, ad esempio GitHub Actions o TravisCI. È possibile lavorare solo in locale, ma un approccio di questo tipo avrà un impatto negativo sulla valutazione finale.

12.6 Attività richieste: generazione dei test

12.6.1 Scelta delle classi

IMPORTANTE PER L'ESAME

Il docente raccomanda esplicitamente di non scegliere classi banali, né classi legate alla GUI. In questi casi sarebbe difficile dimostrare l'apprendimento dei concetti del corso.

Solo nelle slide (non spiegato a voce)

Se si intuisce che una classe raggiungerà subito il 100% di coverage senza sforzo, è consigliato contattare il docente.

12.6.2 Test manuali

Su ciascuna classe bisogna definire un insieme di test tramite **category partition**, adottando preferibilmente un approccio **black-box** basato sulle funzionalità esplicite o implicite della classe.

12.6.3 Test automatici

Occorre poi generare automaticamente altri test usando tre approcci distinti:

- approcci **randomici**;
- approcci basati su **LLM** e sperimentazione di diversi prompt;
- approcci guidati da **coverage control-flow**.

12.6.4 Integrazione nel build

Nota del Prof

Per evitare problemi di configurazione e interferenze, è consigliabile disabilitare o rimuovere tutti i test nativi del progetto Apache, lasciando nel ciclo di build soltanto i test sviluppati da voi.

12.7 Valutazione della qualità dei test

La qualità dei test deve essere validata scegliendo almeno **due metriche di adeguatezza**, non necessariamente calcolate da tool automatici, e confrontando i valori ottenuti.

Occorre inoltre:

- valutare la significatività dei test rispetto alle funzionalità attese;
- migliorare i test manuali aumentando la loro adeguatezza;
- applicare tecniche di **mutation testing**;
- aggiungere nuovi test per migliorare la robustezza della suite;
- stimare la **reliability** del sistema assumendo un profilo operativo uniforme.

Solo nelle slide (non spiegato a voce)

Per consentire il completamento della build, quando necessario è possibile disabilitare temporaneamente i test che falliscono, purché la cosa venga documentata.

Nota del Prof

Se un test fallisce sulla classe originale, ci sono due possibilità: o il test è stato scritto male e quindi va corretto, oppure avete scoperto un bug reale, che va documentato e gestito separatamente.

12.8 Varianti delle classi e uso degli LLM

La parte finale del progetto riguarda l'evoluzione del sistema e il comportamento dei test quando il codice cambia.

A partire dalla classe originale C_0 , bisogna chiedere a un LLM di generare quattro varianti di refactoring:

$$C_1, C_2, C_3, C_4$$

Nota del Prof

Il fulcro dell'esperimento è fornire all'LLM quantità crescenti di informazione. In particolare:

- C_1 : si fornisce solo la classe originale e si chiede il refactoring;
- C_2 : si chiede il refactoring imponendo il passaggio dei test black-box;
- C_3 : si aggiunge anche il vincolo di passare i test di control flow;
- C_4 : si impone il passaggio di tutti i test, inclusa la mutation analysis.

Occorre poi:

- eseguire tutti i test originari sulle varianti;
- verificare se le funzionalità iniziali siano state mantenute;
- generare nuovi test automatici per le varianti;
- confrontare qualitativamente i test delle varianti con quelli della classe originale.

Nota del Prof

Se la variante C_4 non supera un test che era stato fornito esplicitamente come vincolo all'LLM, significa che l'LLM ha fallito il refactoring.

12.9 Report finale e consegna

Lo studente deve redigere un **report dettagliato** in cui descrive:

- le attività svolte;
- le decisioni prese;
- la metodologia seguita;
- i problemi affrontati;
- i risultati ottenuti.

Il report ideale ha una lunghezza di circa **12 pagine A4**, a interlinea singola, carattere 10, font Arial, in formato PDF.

Nota del Prof

Le 12 pagine si riferiscono al testo descrittivo vero e proprio. Figure, tabelle e listati di codice vanno inseriti alla fine, come allegati, e referenziati nel testo.

IMPORTANTE PER L'ESAME

Non bisogna usare figure, tabelle o didascalie come scorciatoia per spostare fuori dal conteggio l'analisi testuale. Le tabelle mostrano i dati; la discussione e l'interpretazione devono stare nel corpo del report.

12.9.1 File `classes.txt`

Va inoltre consegnato un file di testo chiamato `classes.txt`, contenente i nomi completi delle classi considerate, in ordine alfabetico, nel formato:

$$\langle nomePackage \rangle . \langle nomeSubPackage \rangle . \dots \langle nomeClasse \rangle$$

12.9.2 Invio e discussione orale

Il report e il file `classes.txt` vanno inviati via email al docente entro le scadenze pubblicate su Teams. L'esame orale consiste nella presentazione e discussione del report.

Nota del Prof

I link al repository GitHub e al framework di CI dovrebbero essere inseriti esplicitamente nel report oppure nella mail di consegna.

12.10 Messaggi chiave della lezione

IMPORTANTE PER L'ESAME

Il docente valuterà la comprensione critica del lavoro svolto. Non basta riportare un valore numerico, per esempio un coverage del 37%: bisogna spiegare perché si è ottenuto quel valore e quali implicazioni ha.

Nota del Prof

Quando si generano le varianti C_1 – C_4 con l'LLM, conviene sempre iniziare una nuova sessione di chat fresca, in modo da evitare che il modello venga influenzato dai prompt precedenti e produca varianti troppo simili tra loro.

In sintesi

Il modulo di Software Testing prevede un esame basato su un progetto individuale rigoroso. Lo studente deve lavorare su due classi non banali di un progetto Apache, generare e valutare test con approcci diversi, integrare i test in una pipeline di build, studiare l'effetto delle mutazioni e delle varianti generate con LLM, e infine documentare tutto in un report tecnico dettagliato da discutere oralmente.